

Camunda 8 → Temporal Migration Assessment

Generated 2026-08-24 09:17 UTC

1. Executive Summary

Executive Summary

This estate comprises 16 production processes spanning account management, lending, payments, onboarding, and compliance domains. Core workflows include **Account Closure** (event-based gateway racing signals against timers), **Loan Application** (orchestrating multiple sub-processes with cancellation handling), **Fund Disbursement** (saga-pattern compensation for ledger rollback), and **Credit Bureau Assessment** (DMN-driven affordability rules with escalation logic). The estate earns an Automation Readiness score of 45 (High Complexity), driven primarily by 6 user task implementations requiring a custom task service and 5 DMN decision integrations requiring rules-engine work or logic ports. The recommended first step is to prioritize the 12 mechanically translatable workers for automated conversion while our team delivers the custom task service foundation that unblocks **Complaint Handling, Underwriting Review, Document Collection**, and the other user-task-dependent processes.



Key Observations

AI-assisted: the points below are interpreted by AI and may include assumptions where source data was not explicitly provided. Please validate against your environment.

- User task infrastructure will require the most custom implementation effort.** Six processes—**Complaint Handling, Underwriting Review, Document Collection, Identity Verification, Loan Application, and Customer Onboarding & KYC**—rely on user tasks with form-based human interaction. Temporal provides no native task service, so our team will deliver a signal-driven task abstraction that preserves assignment, claiming, and completion semantics. This foundation unlocks the majority of the assisted-rewrite workload.
- DMN decision logic requires integration or porting.** Five business rule tasks across **Credit Bureau Assessment** and **Regulatory Reporting Batch** depend on decision tables for affordability bands and transaction classification. We will either integrate a standalone DMN engine as a callable activity or port high-value rules directly into Java activities, guided by your team's preference for rules governance versus code maintainability.
- Saga-pattern compensation is isolated to one critical process.** **Fund Disbursement** uses BPMN compensation boundaries to roll back ledger reservations on transaction failure. Temporal has no

compensation runtime, so we will implement this as an explicit saga with compensating activities that mirror the reservation-and-rollback logic. This is the estate's only compensation use case, limiting saga-build scope to a single high-value flow.

Signal broadcasting requires custom orchestration. **Base Rate Change Broadcast** throws signals to fan out rate changes across multiple downstream workflows, and **Loan Application** uses message boundaries to handle cancellation requests. Temporal signals are point-to-point, so we will implement broadcast patterns using Temporal queries to discover target workflow IDs or an event bus for pub-sub fan-out. This pattern appears in 2 processes and will be templated for reuse.

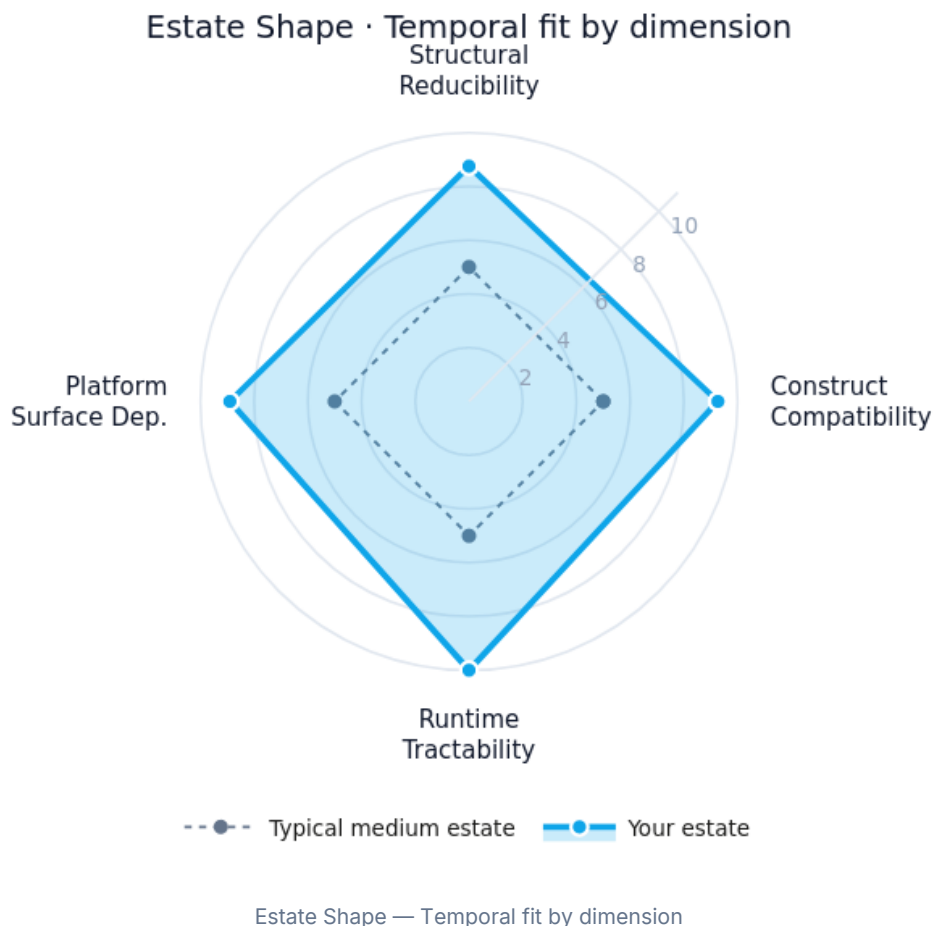
Code coverage shows 12 workers translate mechanically, 22 need engineering review. Of 34 analyzed workers, 12 map directly to Temporal activities with no manual intervention. The remaining 22—covering user tasks, DMN calls, signal handlers, and error-boundary logic—will be migrated using our accelerator tooling to generate scaffolding, with your team completing domain-specific integration under our guidance. No workers require fully bespoke rewrites.

2. What Was Assessed

AI-assisted: the points below are interpreted by AI and may include assumptions where source data was not explicitly provided. Please validate against your environment.

Property	Value
Cluster scan	Yes — version 8.8.34
Process definitions analysed	78
Code analysis (shinro analyze)	Yes — 34 workers
Document analysis	Not provided
Questionnaire	10 of 28 answered

3. Your Camunda Environment



8.0 grade A



This Camunda 8 estate comprises 16 distinct process definitions (78 versions total) running across a single-partition cluster on version 8.8.34 with multi-tenancy enabled for 3 tenants. There are 673 active instances, the longest running for 70 days, and 33 distinct job types distributed across processes. The estate includes 6 user tasks (5 native, 1 job-worker-based) with 100 currently open, 5 DMN decisions, 2 connector types in use, 1 signal throw event, and 3 compensation elements. Process complexity is distributed across 2 high-complexity, 9 medium-complexity, and 5 low-complexity definitions, with an overall estate grade of A and a score of 8.0, reflecting strong construct compatibility and structural reducibility.

The assessment drew on cluster telemetry and static code analysis but did not include document analysis, meaning operational runbooks, architecture diagrams, deployment pipelines, and organisational context were not reviewed. Additionally, 10 of 28 questionnaire items were answered, leaving gaps in understanding around team structure, testing practices, and non-functional requirements. Features such as user tasks, DMN, connectors, signals, and compensation will require custom implementations in Temporal, which our team will deliver as part of the migration service. The partial questionnaire coverage means some operational and organisational dimensions remain to be explored in discovery workshops before migration planning is finalised.

Property	Value
----------	-------

Cluster version	8.8.34
API adapter	orchestration-cluster-v2
Cluster size	1
Partitions	1
Replication factor	1
Multi-tenancy	Enabled
Permission gaps	None

This estate scores **8.0** → grade **A**. The estate grade measures how well the estate's BPMN construct shape maps to Temporal — construct compatibility, control-flow reducibility, platform-surface reliance, and runtime fit. It is a construct-level view only; the Automation Readiness Score (§6) adds code analysis, operational coupling, and systemic risk on top. Constructs map near-directly to Temporal — minimal adaptation needed.

Each dimension is scored 0–1, where **1.00 is best** (closest fit to Temporal).

Dimension	Score (0–1)	Assessment	What it measures
Construct Compatibility	0.93	Low complexity	How many BPMN constructs have a direct Temporal equivalent
Structural Reducibility	0.88	Low complexity	Whether control flow maps to nested blocks, or needs manual decomposition
Platform Surface Dependency	0.89	Low complexity	How little you rely on Tasklist / Forms / DMN — the parts Temporal has none of
Runtime Tractability	1.00	Low complexity	Whether instance volumes and lifetimes fit Temporal's execution model

Operational coupling — instance modification, Optimize reliance, downstream consumers — is **not** an estate-grade dimension. No Camunda endpoint reports it, so it cannot come from the cluster scan; it is scored from your questionnaire answers as factor **F7** in the Automation Readiness section.

User Tasks:

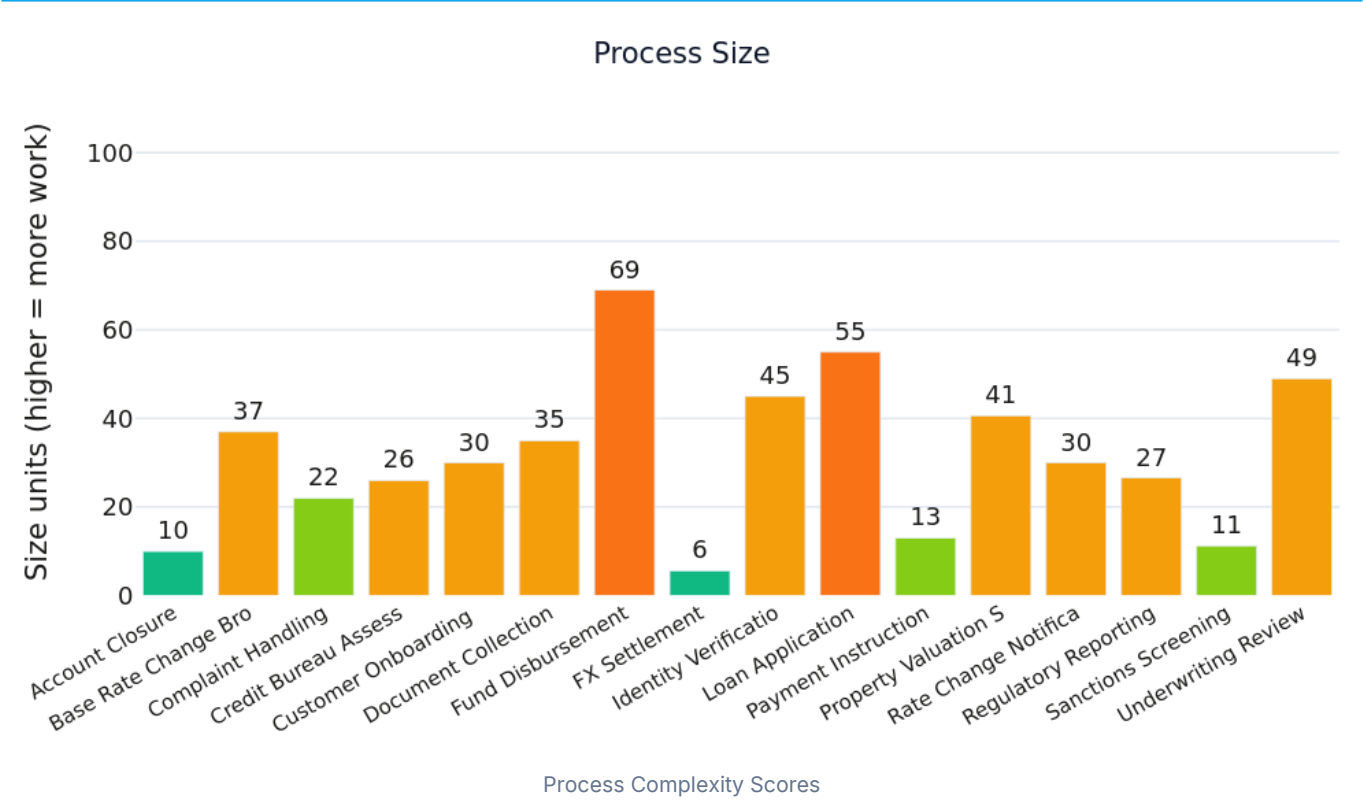
Property	Value
Total user tasks	6
Native (zeebe:userTask)	5
Job-worker style	1
Distinct forms	5
Distinct candidate groups	5
Candidate groups resolved only at runtime (FEEL)	0

Tasks using a due date	1
Tasks using a follow-up date	1
Tasks using a priority	4
Open tasks at scan time	100
Average open-task age	48.6 days

DMN Decisions:

Decision	Latest Ver	Versions	Type	Evaluations	Invoked From
affordability-band	3	3	—	0	credit- bureau- assessment/ br_ credit_risk
fee-waiver	3	3	—	0	—
override-precedence	3	3	—	0	—
pricing-tier	3	3	—	0	—
risk-flags	3	3	—	0	—

4. Process Landscape



AI-assisted: the points below are interpreted by AI and may include assumptions where source data was not explicitly provided. Please validate against your environment.

Account Management — Handles account lifecycle operations including closure workflows that await confirmation or timeout conditions.

Process / ID	Ver	Size	Cplx	Active	Elem	Class	What it does	Notes
Account Closure · account-closure	3	10	Low	0	7	AUTO+FLAG	Closes customer accounts by waiting for either a confirmation message or a timeo	Mechanical port — the event-based gateway maps to a Temporal selector that races a signal against a timer, and the receive task and message catch both become signal handlers.

Product Management — Orchestrates product-related events such as broadcasting base rate changes and notifying customers of interest rate adjustments across the product portfolio.

Process / ID	Ver	Size	Cplx	Active	Elem	Class	What it does	Notes
Base Rate Change Broadcast · base-rate-change-broadcast	3	37	Medium	0	8	MANUAL	Broadcasts central bank base rate changes to affected product books and downstre	Assisted rewrite — the inclusive gateway requires evaluating multiple conditions and fanning out to a dynamic set of activities, and the signal throw needs custom logic to start or signal multiple target workflows.
Rate Change Notification · rate-change-notification	3	30	Medium	0	5	MANUAL	Notifies customers of interest rate changes by identifying affected clients and	Mechanical port — the signal start maps to a signal-triggered workflow, the parallel multi-instance becomes a fan-out of concurrent activities, and service tasks map directly to activities.

Customer Service — Manages customer complaint intake, triage, investigation, and resolution with service level escalation.

Process / ID	Ver	Size	Cplx	Active	Elem	Class	What it does	Notes
--------------	-----	------	------	--------	------	-------	--------------	-------

Complaint Handling · complaint-handling	3	22	Low	10	14	AUTO+ FLAG	Manages customer complaints from intake through triage, investigation, and statu	Custom build — the user task needs a task service with signal completion, the message start and timer start require dual trigger logic, the escalation interrupting event needs a custom escalation workflow, and the non-interrupting event subprocess maps to a side-effect listener.
---	---	----	-----	----	----	------------	--	---

Lending — Manages the complete lending lifecycle from application intake and credit assessment through document collection, property valuation, underwriting review, and fund disbursement.

Process / ID	Ver	Size	Cplx	Active	Elem	Class	What it does	Notes
Credit Bureau Assessment · credit-bureau-assessment	3	26	Medium	100	14	AUTO+ FLAG	Evaluates creditworthiness by calling external credit bureaus, applying affordab	Assisted rewrite — the business rule task requires integrating a DMN decision engine or porting affordability-band logic into code, the signal catch needs a signal handler, and escalation events need custom escalation workflows.
Document Collection · document-collection	3	35	Medium	100	14	AUTO+ FLAG	Gathers required documents from applicants, sends reminders, and manages a user	Custom build — the user task needs a task service abstraction with signal-based completion, the sequential multi-instance becomes a loop over activities, and non-interrupting boundary timers map to side-effect workflows that send reminders without cancelling the main flow.

Loan Application · loan-application	3	55	High	100	18	AUTO+ FLAG	Orchestrates the end-to-end loan application lifecycle, coordinating multiple su	Custom build — the five call activities map to child workflows, the message boundary interrupting event requires a signal handler for cancellation and withdrawal, and the event subprocess needs a Temporal signal listener pattern.
Property Valuation Scheduling · property-valuation-scheduling	3	41	Medium	11	7	MANUAL	Schedules property valuations by coordinating surveyor availability, arranging a	Assisted rewrite — the ad-hoc subprocess requires custom dynamic task orchestration logic, the call activity maps to a child workflow, and the timer start event becomes a cron schedule.
Underwritin g Review · underwritin g- review	3	49	Medium	0	13	AUTO+ FLAG	Performs multi-stage underwriting reviews with SLA timers, escalating breaches a	Custom build — the two user tasks require a task service with signal completion, the event-based gateway maps to a Temporal selector racing a message signal against a timer, and the interrupting boundary timer needs a Temporal timer that cancels the current scope on expiry.

Onboarding — Handles customer onboarding workflows including identity verification and Know Your Customer checks to establish compliant customer relationships.

Process / ID	Ver	Size	Cplx	Active	Elem	Class	What it does	Notes
--------------	-----	------	------	--------	------	-------	--------------	-------

Customer Onboarding & KYC · customer-onboarding-kyc	3	30	Medium	100	7	AUTO+ FLAG	Onboards new customers by running parallel KYC checks via call activities and wa	Assisted rewrite — the call activities map to child workflows, the parallel multi-instance becomes a fan-out of concurrent child workflows, the computed message catch requires a dynamic signal name pattern, and the user task needs a task service abstraction.
Identity Verification · identity-verification	3	45	Medium	100	10	AUTO+ FLAG	Confirms customer identity by querying identity bureaus, handling errors via ret	Custom build — the user task requires a task service with signal-based completion, the error boundary maps to Temporal retry policies or compensating activities, and the non-interrupting message boundary needs a side-effect signal listener.

Payments — Processes payment transactions including foreign exchange settlement, fund disbursement with ledger compensation, and payment rail instruction with retry logic.

Process / ID	Ver	Size	Cplx	Active	Elem	Class	What it does	Notes
Fund Disbursement · fund-disbursement	3	69	High	0	13	AUTO+ FLAG	Disburses loan funds by reserving amounts, posting ledger entries, notifying dow	Custom build — the compensation boundary, handler, and throw events require implementing Temporal saga pattern with compensating activities, and the message throw and send task need signal or update emission to external workflows.
FX Settlement · fx-settlement	9	6	Low	42	6	AUTO	Settles foreign exchange transactions by locking rates, executing settlement leg	Mechanical port — all four service tasks map directly to activities in a linear workflow with no special constructs.

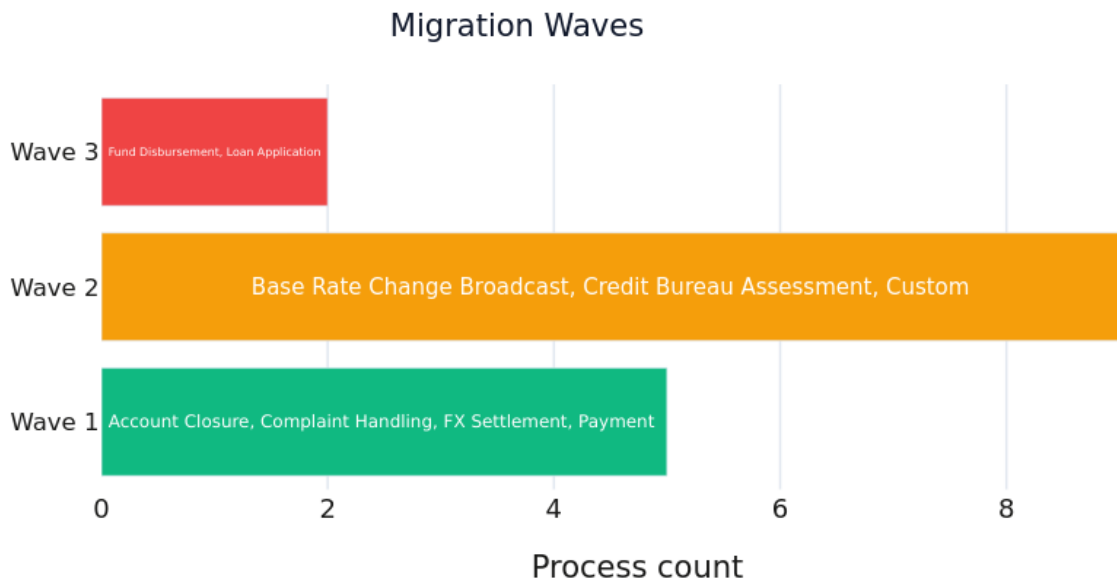
Payment Instruction · payment-instruction	3	13	Low	0	15	AUTO+FLAG	Executes payment instructions by submitting to a payment rail, retrying via alte	Assisted rewrite — the error event subprocess maps to a Temporal error handler or compensating workflow, and the link events require restructuring control flow into explicit activity sequences or continue-as-new patterns.
---	---	----	-----	---	----	-----------	--	---

Compliance — Executes compliance obligations including sanctions and politically exposed persons screening and periodic regulatory reporting to meet legal and supervisory requirements.

Process / ID	Ver	Size	Cplx	Active	Elem	Class	What it does	Notes
Regulatory Reporting Batch · regulatory-reporting-batch	3	27	Medium	10	7	AUTO+FLAG	Generates periodic regulatory reports by building datasets, classifying transact	Assisted rewrite — the business rule task and script tasks require porting DMN and script logic into activities or integrating a rules engine, the parallel multi-instance becomes fan-out child workflows, and the timer start maps to a cron schedule.
Sanctions Screening · sanctions-screening	3	11	Low	100	6	AUTO+FLAG	Screens customers against sanctions and politically exposed persons lists, trigg	Mechanical port — the timer start event maps to a cron schedule or scheduled workflow, service tasks map directly to activities, and the signal catch becomes a signal handler that starts or continues the workflow.

Classification key — [AUTO](#): translates mechanically to Temporal, no design decisions · [AUTO+FLAG](#): translates, but a human must confirm one specific choice · [MANUAL](#): no mechanical equivalent — needs bespoke implementation.

5. Migration Strategy



Migration Waves

AI-assisted: the points below are interpreted by AI and may include assumptions where source data was not explicitly provided. Please validate against your environment.

This estate comprises 16 processes spanning 7 domains, with a mix of orchestration patterns and human-task coordination. The dominant migration classification is **assisted rewrite** (10 processes), followed by custom build (4 processes) and mechanical port (2 processes). Mechanical ports include FX Settlement and Sanctions Screening, which map directly to Temporal activities and cron schedules. Assisted rewrites include Base Rate Change Broadcast, Credit Bureau Assessment, Customer Onboarding & KYC, Document Collection, Payment Instruction, Property Valuation Scheduling, Rate Change Notification, and Regulatory Reporting Batch—all requiring DMN integration, dynamic task orchestration, or parallel child workflow patterns. Custom builds include Complaint Handling, Fund Disbursement, Identity Verification, Loan Application, and Underwriting Review, where user tasks, compensation sagas, and event-driven escalation require bespoke Temporal signal and task-service implementations.

The recommended phased wave plan prioritizes low-risk value delivery. **Wave 1** begins with FX Settlement and Sanctions Screening—two mechanical ports requiring no custom patterns, delivering early confidence and establishing the deployment pipeline. **Wave 2** tackles assisted rewrites: Rate Change Notification and Base Rate Change Broadcast (signal-triggered fan-outs), followed by Credit Bureau Assessment and Regulatory Reporting Batch (DMN and business rule integration), then Payment Instruction and Property Valuation Scheduling (error handling and ad-hoc orchestration). **Wave 3** addresses custom builds: Document Collection and Identity Verification (user task and timer patterns), Fund Disbursement (compensation saga), Complaint Handling (escalation and event subprocess), Underwriting Review (event-based gateway and SLA timers), and finally Loan Application (multi-child-workflow orchestration with cancellation).

The single highest-leverage first step is **migrating FX Settlement in Wave 1**. This low-complexity process exercises the full Temporal deployment, monitoring, and rollback mechanics with zero custom pattern risk. A successful FX Settlement migration validates the infrastructure, trains the team on Temporal deployment, and builds stakeholder confidence before tackling the 5 DMN decisions, 6 user tasks, and 3 compensation handlers waiting in later waves. It also allows early testing of the worker migration patterns that will repeat across 22 assisted-rewrite workers in Wave 2.

MVP candidates — lowest-complexity processes first:

- **FX Settlement** — low complexity, 6 size units, 6 elements.
- **Account Closure** — low complexity, 10 size units, 7 elements.

- **Sanctions Screening** — low complexity, 11 size units, 6 elements.

Waves:

Wave	Processes	Count
Wave 1 — Mechanical port	Account Closure, Complaint Handling, FX Settlement, Payment Instruction, Sanctions Screening	5
Wave 2 — Assisted	Base Rate Change Broadcast, Credit Bureau Assessment, Customer Onboarding & KYC, Document Collection, Identity Verification, Property Valuation Scheduling, Rate Change Notification, Regulatory Reporting Batch, Underwriting Review	9
Wave 3 — Custom build	Fund Disbursement, Loan Application	2

6. Automation Readiness

Overall: **45 / 100** (High Complexity).

Score = 100 – 55 penalty points across the eight factors below.

Factor	Penalty	Reason
F1 · Process Complexity	9 / 20	Construct compatibility score 0.9 — some BPMN constructs need custom Temporal patterns; Structu...
F2 · Worker Migration Coverage	9 / 15	22 worker(s) classified ASSISTED — need review and partial rewrite
F3 · Job Type Risk	9 / 15	4 side-effecting worker(s) — need careful retry policy design; 5 worker(s) with autoComplete=fa...
F4 · User Task / Signal Gap	11 / 15	6 user task(s) in BPMN — need Temporal Signal/Update handlers for human interaction; 5 distinct...
F5 · Timer / Event Complexity	7 / 10	2 non-interrupting boundary timer(s) — need Temporal timer + signal pattern; 3 unbounded cycle(...
F6 · Estate Scale	10 / 10	673 active instance(s) — drain window needed before cutover; 5 DMN decision(s) — need Temporal...
F7 · Operational Coupling	0 / 5	Not assessed — the questionnaire answers that determine operational coupling were not available...
F8 · Systemic Risk	0 / 10	No estate-wide blocker — at least some processes/workers can be ported independently
Total	55 / 100	

7. Risks & Mitigations

AI-assisted: the points below are interpreted by AI and may include assumptions where source data was not explicitly provided. Please validate against your environment.

Risk Flag	Count	Temporal Approach
Non-interrupting boundary timers	2	Timer + signal pattern
Unbounded cycles	3	Workflow loop with continuation check
Cron with non-zero seconds	1	Temporal cron schedule (minute precision)
Durations over 30 days	1	Long-running workflow with checkpoint

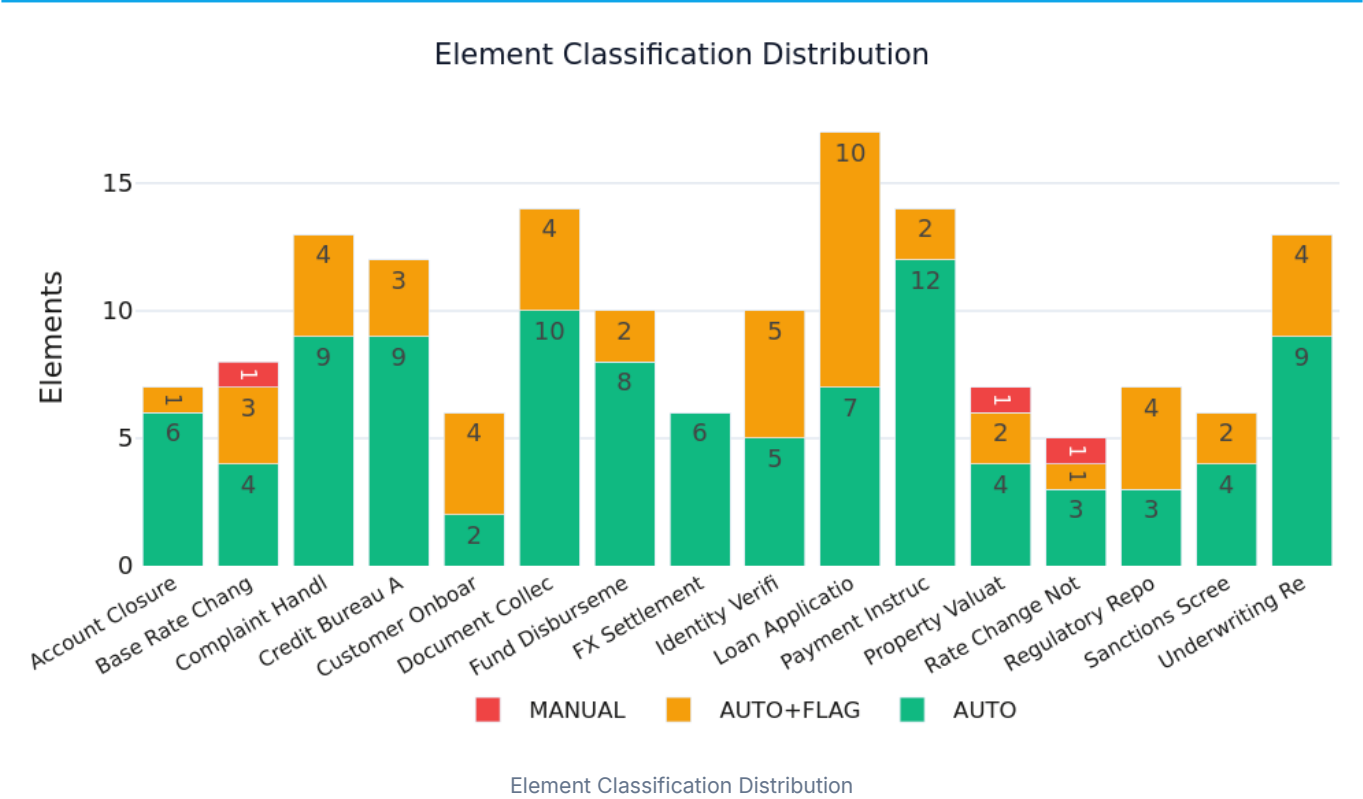
Consideration	Business Impact	Effort	Mitigation
User task signal handlers (Complaint Handling, Document Collection, Identity Verification, Underwriting Review, Loan Application)	Manual steps in complaints, onboarding, and underwriting require human input; forms need Temporal signal/update integration	High	Our team will implement a task-service abstraction and signal-based completion handlers for all 6 user tasks and 5 distinct forms, with reference patterns for each process
DMN decision integration (Credit Bureau Assessment, Regulatory Reporting Batch)	Affordability-band and transaction-classification rules need equivalent logic in Temporal	Medium	We will port the 5 DMN decisions into Temporal activities or integrate a Java decision engine, delivering a reusable pattern for business rules
Compensation saga (Fund Disbursement)	Ledger reservation failures require rollback; missing this risks duplicate disbursements or orphaned reserves	High	Our team will implement Temporal saga pattern with compensating activities for the 3 compensation elements, ensuring idempotent rollback
673 active instances (estate-wide)	Cutover requires draining in-flight workflows or dual-running Camunda and Temporal during transition	Medium	We will design a phased cutover plan with instance-completion monitoring, migration scripts for long-running workflows, and a controlled drain window
Non-interrupting boundary timers (Document Collection, Identity Verification)	Reminder and chase logic must not cancel main workflow; Temporal timer patterns differ from BPMN side effects	Medium	Our team will implement Temporal timer-plus-signal patterns for the 2 non-interrupting boundaries, with reference code for side-effect workflows

Event-based gateway selector (Account Closure, Underwriting Review)	Racing a signal against a timer requires Temporal selector pattern; incorrect implementation risks deadlock	Low	We will deliver Temporal selector logic for the 2 event-based gateways, with test harnesses for signal-vs-timer race conditions
--	---	-----	---

Top code-analysis gaps:

- `no-test-oracles` — no input→output test oracles were extracted; generated code has no dual-run seed from existing tests

8. Process Detail Cards



Account Closure

Elements:

Element Type	Count	Class	Findings
<code>endEvent.none</code>	2	AUTO	—
<code>gateway.eventBased</code>	1	AUTO	—
<code>message.intermediateCatch.simple</code>	1	AUTO	—
<code>receiveTask</code>	1	AUTO+FLAG	—
<code>startEvent.none</code>	1	AUTO	—
<code>timer.intermediateCatch</code>	1	AUTO	—

Classification key — [AUTO](#): translates mechanically to Temporal, no design decisions · [AUTO+FLAG](#): translates, but a human must confirm one specific choice · [MANUAL](#): no mechanical equivalent — needs bespoke implementation.

Control flow diagram:

Control-flow diagram available in the interactive report — see the canvas panel.

Base Rate Change Broadcast

Elements:

Element Type	Count	Class	Findings
endEvent.terminate	1	AUTO+FLAG	—
gateway.inclusive	2	AUTO+FLAG	—
serviceTask	3	AUTO	—
signal.throw	1	MANUAL	—
startEvent.none	1	AUTO	—

Classification key — [AUTO](#): translates mechanically to Temporal, no design decisions · [AUTO+FLAG](#): translates, but a human must confirm one specific choice · [MANUAL](#): no mechanical equivalent — needs bespoke implementation.

Control flow diagram:

Control-flow diagram available in the interactive report — see the canvas panel.

Job types: [flag-product-book](#)

Complaint Handling

Elements:

Element Type	Count	Class	Findings
endEvent.none	3	AUTO	—
escalation.interrupting	1	AUTO+FLAG	—
gateway.exclusive	1	AUTO+FLAG	—
manualTask	1	AUTO	—
message.start	1	AUTO	—
serviceTask	1	AUTO	—
startEvent.none	1	AUTO	—
subprocess.embedded	1	AUTO	—
subprocess.eventNonInterrupting	1	ASSISTED	—
timer.start	1	AUTO+FLAG	—

undefinedTask	1	AUTO	—
userTask	1	AUTO+FLAG	—

Classification key — [AUTO](#): translates mechanically to Temporal, no design decisions · [AUTO+FLAG](#): translates, but a human must confirm one specific choice · [MANUAL](#): no mechanical equivalent — needs bespoke implementation.

Control Flow Graph:

Property	Value
Implicit joins (no gateway)	1

- [sub_investigate](#): 2 incoming flows converge here with no joining gateway — BPMN gives a subProcess implicit XOR-join semantics, so it runs once per arriving token rather than waiting for the others

Control flow diagram:

Control-flow diagram available in the interactive report — see the canvas panel.

Job types: [send-complaint-status-update](#)

Forms: complaint-triage-form

Credit Bureau Assessment

Elements:

Element Type	Count	Class	Findings
businessRuleTask.inBroker	1	ASSISTED	—
endEvent.none	3	AUTO	—
escalation.interrupting	1	AUTO+FLAG	—
escalation.nonInterrupting	1	ASSISTED	—
gateway.exclusive	1	AUTO+FLAG	—
serviceTask	3	AUTO	—
signal.catch	1	AUTO+FLAG	—
startEvent.none	2	AUTO	—
subprocess.embedded	1	AUTO	—

Classification key — [AUTO](#): translates mechanically to Temporal, no design decisions · [AUTO+FLAG](#): translates, but a human must confirm one specific choice · [MANUAL](#): no mechanical equivalent — needs bespoke implementation.

Control Flow Graph:

Property	Value
Implicit joins (no gateway)	1

- `catch_rate_change`: 2 incoming flows converge here with no joining gateway — BPMN gives a `signalCatchEvent` implicit XOR-join semantics, so it runs once per arriving token rather than waiting for the others

Control flow diagram:

Control-flow diagram available in the interactive report — see the canvas panel.

Job types: `notify-senior-analyst`

DMN decisions: affordability-band

Connectors (Camunda-runtime — not job workers):

Type	Count
core-banking-api-call	1
io.camunda:http-json:1	1

Customer Onboarding & KYC

Elements:

Element Type	Count	Class	Findings
<code>callActivity</code>	2	AUTO+FLAG	—
<code>endEvent.none</code>	1	AUTO	—
<code>message.intermediateCatch.computed</code>	1	ASSISTED	—
<code>multiInstance.parallel</code>	1	AUTO+FLAG	collection size at runtime is unknown from static BPMN alone — doc 02 §5.24 recommends batched child workflows once a collection nears ~500 items; cross-check against row 22's variable payload samples for this collection if available.
<code>startEvent.none</code>	1	AUTO	—
<code>userTask</code>	1	AUTO+FLAG	—

Classification key — `AUTO`: translates mechanically to Temporal, no design decisions · `AUTO+FLAG`: translates, but a human must confirm one specific choice · `MANUAL`: no mechanical equivalent — needs bespoke implementation.

Data flow findings:

- Call activity "call_verify_per_applicant" → "identity-verification": approximate true input set is applicantRef, requiredDocuments — cross-check against this call site's own ioMapping before porting
- Element "call_verify_per_applicant" input mapping to "applicantRef" is a non-trivial FEEL expression (=applicant.ref) — translation risk, see doc 07

Control flow diagram:

Control-flow diagram available in the interactive report — see the canvas panel.

Document Collection

Elements:

Element Type	Count	Class	Findings
<code>endEvent.none</code>	3	AUTO	—
<code>message.intermediateCatch.simple</code>	1	AUTO	—
<code>multiInstance.sequential</code>	1	AUTO	—
<code>receiveTask</code>	1	AUTO+FLAG	—
<code>serviceTask</code>	3	AUTO	—
<code>startEvent.none</code>	1	AUTO	—
<code>timer.boundaryNonInterrupting</code>	2	AUTO+FLAG	—
<code>timer.intermediateCatch</code>	1	AUTO	—
<code>userTask</code>	1	AUTO+FLAG	—

Classification key — [AUTO](#): translates mechanically to Temporal, no design decisions · [AUTO+FLAG](#): translates, but a human must confirm one specific choice · [MANUAL](#): no mechanical equivalent — needs bespoke implementation.

Control flow diagram:

Control-flow diagram available in the interactive report — see the canvas panel.

Job types: [chase-document](#), [send-document-reminder](#)

Forms: document-upload-form

Fund Disbursement

Elements:

Element Type	Count	Class	Findings
<code>callActivity</code>	1	AUTO+FLAG	—
<code>compensation.boundary</code>	1	ASSISTED	—
<code>compensation.handler</code>	1	ASSISTED	—
<code>compensation.throwOrEnd</code>	1	ASSISTED	—
<code>endEvent.none</code>	2	AUTO	—
<code>gateway.exclusive</code>	1	AUTO+FLAG	—
<code>message.throw</code>	1	AUTO	—
<code>sendTask</code>	1	AUTO	—

serviceTask	3	AUTO	—
startEvent.none	1	AUTO	—

Classification key — [AUTO](#): translates mechanically to Temporal, no design decisions · [AUTO+FLAG](#): translates, but a human must confirm one specific choice · [MANUAL](#): no mechanical equivalent — needs bespoke implementation.

Control flow diagram:

Control-flow diagram available in the interactive report — see the canvas panel.

Job types: [manual-ledger-adjustment](#), [notify-ledger-posted](#), [release-fund-reservation](#), [reserve-funds](#)

FX Settlement

Elements:

Element Type	Count	Class	Findings
endEvent.none	1	AUTO	—
serviceTask	4	AUTO	—
startEvent.none	1	AUTO	—

Classification key — [AUTO](#): translates mechanically to Temporal, no design decisions · [AUTO+FLAG](#): translates, but a human must confirm one specific choice · [MANUAL](#): no mechanical equivalent — needs bespoke implementation.

Control flow diagram:

Control-flow diagram available in the interactive report — see the canvas panel.

Job types: [confirm-fx-settlement](#), [lock-fx-rate](#), [reconcile-fx-ledger](#), [settle-fx-leg](#)

Identity Verification

Elements:

Element Type	Count	Class	Findings
callActivity	1	AUTO+FLAG	—
endEvent.none	2	AUTO	—
error.boundary	1	AUTO+FLAG	—
gateway.exclusive	1	AUTO+FLAG	—
message.boundaryNonInterrupting	1	AUTO+FLAG	—
serviceTask	2	AUTO	—
startEvent.none	1	AUTO	—
userTask	1	AUTO+FLAG	—

Classification key — [AUTO](#): translates mechanically to Temporal, no design decisions · [AUTO+FLAG](#): translates, but a human must confirm one specific choice · [MANUAL](#): no mechanical equivalent — needs bespoke implementation.

Data flow findings:

- Call activity "call_documents" → "document-collection": approximate true input set is applicationId, requiredDocuments — cross-check against this call site's own ioMapping before porting

Control Flow Graph:

Property	Value
Unmatched split/joins	1
Implicit joins (no gateway)	2

- [gw_result](#): branches converge at 2 different points instead of one common join
- [call_documents](#): 2 incoming flows converge here with no joining gateway — BPMN gives a callActivity implicit XOR-join semantics, so it runs once per arriving token rather than waiting for the others
- [ut_manual_id_review](#): 2 incoming flows converge here with no joining gateway — BPMN gives a userTask implicit XOR-join semantics, so it runs once per arriving token rather than waiting for the others

Control flow diagram:

Control-flow diagram available in the interactive report — see the canvas panel.

Job types: [check-identity-bureau](#), [record-supplementary-evidence](#)

Forms: identity-manual-review-form

Loan Application

Elements:

Element Type	Count	Class	Findings
callActivity	5	AUTO+FLAG	—
endEvent.none	1	AUTO	—
endEvent.terminate	3	AUTO+FLAG	—
gateway.exclusive	1	AUTO+FLAG	—
gateway.parallel	2	AUTO	—
message.boundaryInterrupting	1	AUTO+FLAG	—
message.start	1	AUTO	—
serviceTask	2	AUTO	—
startEvent.none	1	AUTO	—
subprocess.eventInterrupting	1	ASSISTED	—

Classification key — [AUTO](#): translates mechanically to Temporal, no design decisions · [AUTO+FLAG](#): translates, but a human must confirm one specific choice · [MANUAL](#): no mechanical equivalent — needs bespoke implementation.

Data flow findings:

- Call activity "call_disbursement" → "fund-disbursement": approximate true input set is applicationId, approvedAmount — cross-check against this call site's own ioMapping before porting
- Call activity "call_documents" → "document-collection": approximate true input set is applicationId, requiredDocuments — cross-check against this call site's own ioMapping before porting
- Call activity "call_onboarding" → "customer-onboarding-kyc": approximate true input set is applicants, applicationId — cross-check against this call site's own ioMapping before porting
- Call activity "call_underwriting" → "=underwritingProcessId": true child input set could not be resolved (calledProcessId is a FEEL expression (dynamic dispatch) — cannot statically resolve the child) — verify the variables it needs by hand

Control flow diagram:

Control-flow diagram available in the interactive report — see the canvas panel.

Job types: [process-cancellation](#), [process-withdrawal](#)

Payment Instruction

Elements:

Element Type	Count	Class	Findings
endEvent.none	3	AUTO	—
error.end	1	AUTO	—
error.eventSubprocessStart	1	AUTO+FLAG	—
gateway.exclusive	1	AUTO+FLAG	—
linkEvent	2	AUTO	—
serviceTask	3	AUTO	—
startEvent.none	2	AUTO	—
subprocess.embedded	1	AUTO	—
subprocess.eventInterrupting	1	ASSISTED	—

Classification key — [AUTO](#): translates mechanically to Temporal, no design decisions · [AUTO+FLAG](#): translates, but a human must confirm one specific choice · [MANUAL](#): no mechanical equivalent — needs bespoke implementation.

Control flow diagram:

Control-flow diagram available in the interactive report — see the canvas panel.

Job types: [flag-payment-for-review](#), [retry-alternate-rail](#), [submit-payment-rail](#)

Property Valuation Scheduling

Elements:

Element Type	Count	Class	Findings
<code>adHocSubprocess</code>	1	MANUAL	—
<code>callActivity</code>	1	AUTO+FLAG	—
<code>endEvent.none</code>	1	AUTO	—
<code>serviceTask</code>	3	AUTO	—
<code>timer.start</code>	1	AUTO+FLAG	—

Classification key — [AUTO](#): translates mechanically to Temporal, no design decisions · [AUTO+FLAG](#): translates, but a human must confirm one specific choice · [MANUAL](#): no mechanical equivalent — needs bespoke implementation.

Data flow findings:

- Call activity "call_documents" → "document-collection": approximate true input set is applicationId, requiredDocuments — cross-check against this call site's own ioMapping before porting

Control flow diagram:

Control-flow diagram available in the interactive report — see the canvas panel.

Job types: [arrange-property-access](#), [book-surveyor-slot](#), [notify-valuation-appointment](#)

Rate Change Notification

Elements:

Element Type	Count	Class	Findings
<code>endEvent.none</code>	1	AUTO	—
<code>multiInstance.parallel</code>	1	AUTO+FLAG	collection size at runtime is unknown from static BPMN alone — doc 02 §5.24 recommends batched child workflows once a collection nears ~500 items; cross-check against row 22's variable payload samples for this collection if available.
<code>serviceTask</code>	2	AUTO	—
<code>signal.start</code>	1	MANUAL	—

Classification key — [AUTO](#): translates mechanically to Temporal, no design decisions · [AUTO+FLAG](#): translates, but a human must confirm one specific choice · [MANUAL](#): no mechanical equivalent — needs bespoke implementation.

Data flow findings:

- Element "mi_notify" is a parallel multi-instance with no output mapping — Camunda's documented race condition here is silently removed on a mechanical port

Control flow diagram:

Control-flow diagram available in the interactive report — see the canvas panel.

Job types: `find-affected-clients`, `notify-customer`

Regulatory Reporting Batch

Elements:

Element Type	Count	Class	Findings
<code>businessRuleTask.jobWorker</code>	1	AUTO	—
<code>callActivity</code>	1	AUTO+FLAG	—
<code>endEvent.none</code>	1	AUTO	—
<code>multiInstance.parallel</code>	1	AUTO+FLAG	collection size at runtime is unknown from static BPMN alone — doc 02 §5.24 recommends batched child workflows once a collection nears ~500 items; cross-check against row 22's variable payload samples for this collection if available.
<code>scriptTask.inBroker</code>	1	AUTO+FLAG	FEEL expression complexity not analyzed (doc 07's FEEL→Go translator isn't built) — a complex expression may need ASSISTED review rather than a mechanical AUTO+FLAG pass.
<code>scriptTask.jobWorker</code>	1	AUTO	—
<code>timer.start</code>	1	AUTO+FLAG	—

Classification key — `AUTO`: translates mechanically to Temporal, no design decisions · `AUTO+FLAG`: translates, but a human must confirm one specific choice · `MANUAL`: no mechanical equivalent — needs bespoke implementation.

Data flow findings:

- Element "mi_submit_per_currency" is a parallel multi-instance with no output mapping — Camunda's documented race condition here is silently removed on a mechanical port

Control flow diagram:

Control-flow diagram available in the interactive report — see the canvas panel.

Job types: `build-regulatory-dataset`, `classify-transactions-dmn`

Sanctions Screening

Elements:

Element Type	Count	Class	Findings
<code>endEvent.none</code>	1	AUTO	—
<code>serviceTask</code>	2	AUTO	—

signal.catch	1	AUTO+FLAG	—
startEvent.none	1	AUTO	—
timer.start	1	AUTO+FLAG	—

Classification key — [AUTO](#): translates mechanically to Temporal, no design decisions · [AUTO+FLAG](#): translates, but a human must confirm one specific choice · [MANUAL](#): no mechanical equivalent — needs bespoke implementation.

Control flow diagram:

Control-flow diagram available in the interactive report — see the canvas panel.

Job types: [rescreen-sanctions-pep](#), [screen-sanctions-pep](#)

Underwriting Review

Elements:

Element Type	Count	Class	Findings
endEvent.none	3	AUTO	—
gateway.eventBased	1	AUTO	—
gateway.exclusive	1	AUTO+FLAG	—
message.intermediateCatch.simple	1	AUTO	—
serviceTask	2	AUTO	—
startEvent.none	1	AUTO	—
timer.boundaryInterrupting	1	AUTO+FLAG	—
timer.intermediateCatch	1	AUTO	—
userTask	2	AUTO+FLAG	—

Classification key — [AUTO](#): translates mechanically to Temporal, no design decisions · [AUTO+FLAG](#): translates, but a human must confirm one specific choice · [MANUAL](#): no mechanical equivalent — needs bespoke implementation.

Control Flow Graph:

Property	Value
Unmatched split/joins	1
Gateways that both join and fork	1

- [gw_four_eyes](#): branches converge at "gw_wait_outcome", a eventBasedGateway, not a matching exclusiveGateway join
- [gw_wait_outcome](#): gateway joins 2 incoming flows and forks into 2 outgoing ones — it plays both roles at once, so its split/join pairing is ambiguous; model it as a join gateway followed by a separate split

Control flow diagram:

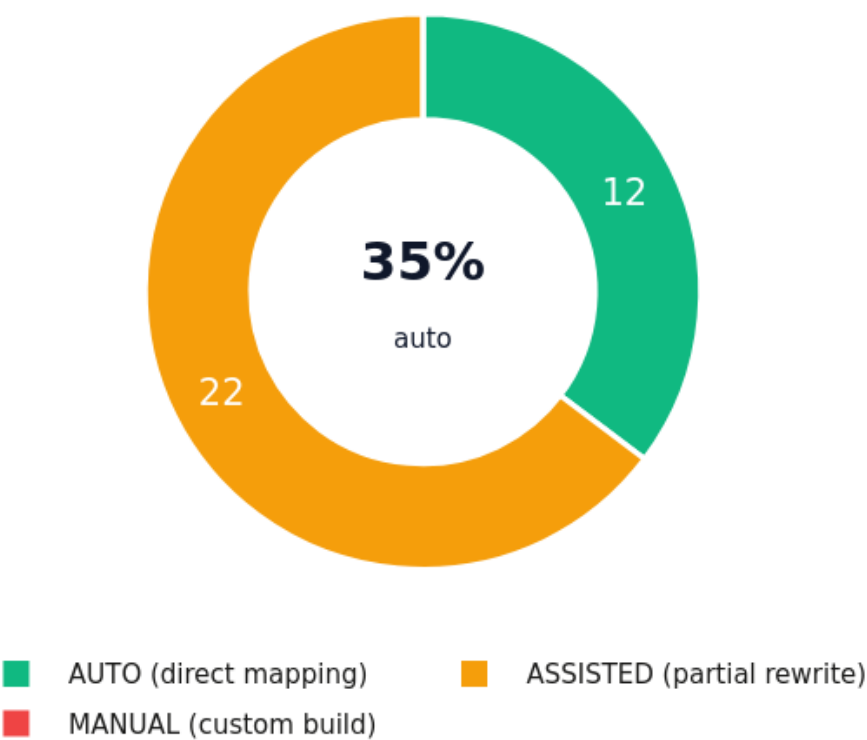
Control-flow diagram available in the interactive report — see the canvas panel.

Job types: [escalate-sla-breach](#), [handle-case-sla-breach](#)

Forms: [underwriting-first-review-form](#), [underwriting-second-review-form](#)

9. Worker Inventory

Worker Migration Coverage



Worker Migration Coverage

34 worker(s) — 22 need a human decision, 12 translate mechanically. Sorted by attention required; expand any row for the full analysis.

Job Type	Coverage	Side Effect	Auto-complete	Flags
retry-alternate-rail	ASSISTED	non-idempotent	No	unsafe to retry, manual complete, throws BPMN error
notify-ledger-posted	ASSISTED	non-idempotent	Yes	unsafe to retry
publish-ledger-posted-message	ASSISTED	non-idempotent	Yes	unsafe to retry
settle-fx-leg	ASSISTED	non-idempotent	Yes	unsafe to retry

check-identity-bureau	ASSISTED	idempotent	No	manual complete, throws BPMN error
lock-fx-rate	ASSISTED	idempotent- with-key	No	manual complete
reserve-funds	ASSISTED	idempotent- with-key	No	manual complete
submit-payment-rail	ASSISTED	idempotent- with-key	No	manual complete, throws BPMN error
arrange-property-access	ASSISTED	idempotent- with-key	Yes	—
book-surveyor-slot	ASSISTED	idempotent- with-key	Yes	—
build-regulatory-dataset	ASSISTED	idempotent	Yes	—
core-banking-api-call	ASSISTED	idempotent- with-key	Yes	—
find-affected-clients	ASSISTED	read-only	Yes	—
notify-customer	ASSISTED	idempotent- with-key	Yes	—
notify-senior-analyst	ASSISTED	idempotent- with-key	Yes	—
notify-valuation-appointment	ASSISTED	idempotent- with-key	Yes	—
record- supplementary-evidence	ASSISTED	idempotent- with-key	Yes	—
release-fund-reservation	ASSISTED	idempotent- with-key	Yes	—
rescreen-sanctions-pep	ASSISTED	idempotent- with-key	Yes	—
screen-sanctions-pep	ASSISTED	idempotent	Yes	—
send-complaint-status-update	ASSISTED	idempotent- with-key	Yes	—
send-document-reminder	ASSISTED	idempotent- with-key	Yes	—
chase-document	AUTO	read-only	Yes	—
classify-transactions-dmn	AUTO	read-only	Yes	—
confirm-fx-settlement	AUTO	read-only	Yes	—

escalate-sla-breach	AUTO	read-only	Yes	—
flag-payment-for-review	AUTO	read-only	Yes	—
flag-product-book	AUTO	read-only	Yes	—
handle-case-sla-breach	AUTO	read-only	Yes	—
legacy-fx-reconcile	AUTO	read-only	Yes	—
manual-onboarding-review	AUTO	read-only	Yes	—
process-cancellation	AUTO	read-only	Yes	—
process-withdrawal	AUTO	read-only	Yes	—
reconcile-fx-ledger	AUTO	read-only	Yes	—

Details — [retry-alternate-rail](#)

What it does	Attempts payment via an alternate rail and throws INSUFFICIENT_FUNDS error if all rails are exhausted.
Language / SDK	java · spring-zeebe
Temporal Activity	Retry-alternate-rail
Registered as	<code>@JobWorker(type = "retry-alternate-rail", autoComplete = false)</code>
Side effect	non-idempotent (<i>llm, confidence 0.75</i>)
Why	The worker calls payment Rail Service. <code>retry Alternate Rail(applicationId, amount)</code> , which strongly suggests initiating a payment transaction on an external payment rail. The code slice shows no idempotency key being read or passed to the rail service. While 'applicationId' is read, it is not used as a deduplication key in the visible code—it is only passed as a parameter. The worker completes with 'paymentStatus' on success or throws INSUFFICIENT_FUNDS on exhaustion. Without evidence that <code>retryAlternateRail</code> internally deduplicates or that the external payment system guarantees idempotency, this must be classified as non-idempotent.
Suggested idempotency key	<code>applicationId</code>
Suggested retry policy	non-retryable: <code>INSUFFICIENT_FUNDS</code>
Reads	<code>applicationId: string</code>
Writes	<code>paymentStatus: string</code>
Call graph	<code>Payment Rail Service. retry Alternate Rail, Logger.warn</code>

Error mapping	<code>PaymentRailService</code> thrown in <code>Payment Rail Service</code> . <code>retry Alternate Rail</code> → <code>INSUFFICIENT_FUNDS</code>
Error mapping	<code>Alternate Rail Exhausted Exception</code> thrown in <code>Payment Rail Service</code> . <code>retry Alternate Rail</code> → <code>INSUFFICIENT_FUNDS</code>
<code>newCompleteCommand</code> → Go	<code>return output, nil</code>
<code>newThrowErrorCommand</code> → Go	<code>Non Retryable Application Error(type="INSUFFICIENT_FUNDS")</code>

Details — [notify-ledger-posted](#)

What it does	Notifies an external system that a ledger has been posted for a given application
Language / SDK	java · spring-zeebe
Temporal Activity	Notify-ledger-posted
Registered as	<code>@JobWorker(type = "notify-ledger-posted")</code>
Side effect	non-idempotent (<i>llm, confidence 0.75</i>)
Why	The worker calls Disbursement Service. <code>notify Ledger Posted</code> with <code>applicationId</code> , which is flagged as reaching an I/O sink. This is a notification operation to an external system. Without visibility into whether the downstream service deduplicates notifications by <code>applicationId</code> or whether the notification is append-only (event log style), I cannot confirm idempotency. The code slice shows only a single outbound call with no retry guards, transaction IDs, or other idempotency mechanisms visible at this layer.
Suggested idempotency key	<code>applicationId</code>
Reads	<code>applicationId</code> : unknown
Call graph	Disbursement Service. notify Ledger Posted
External call	Disbursement Service. notify Ledger Posted (service-call)

Details — [publish-ledger-posted-message](#)

What it does	Notifies an external system that a ledger entry has been posted for a given application
Language / SDK	java · spring-zeebe
Temporal Activity	Publish-ledger-posted-message
Registered as	<code>@JobWorker(type = "publish- ledger- posted- message")</code>
Side effect	non-idempotent (<i>llm, confidence 0.75</i>)

Why	The worker calls Disbursement Service. notify Ledger Posted with applicationId, which is flagged as reaching an I/O sink. The code slice shows only the notification call with no guards, deduplication logic, or idempotency key construction visible. The method name 'notify' suggests a side-effecting operation (likely publishing a message or webhook). Without evidence of idempotency handling in the code slice or call graph, this must be classified as non-idempotent.
Suggested idempotency key	<code>applicationId</code>
Reads	<code>applicationId: unknown</code>
Call graph	<code>Disbursement Service. notify Ledger Posted</code>
External call	<code>Disbursement Service. notify Ledger Posted</code> (service-call)

Details — `settle-fx-leg`

What it does	Settles one leg of a foreign exchange transaction by calling an external settlement service.
Language / SDK	java · spring-zeebe
Temporal Activity	<code>Settle-fx-leg</code>
Registered as	<code>@JobWorker(type = "settle-fx-leg")</code>
Side effect	non-idempotent (<i>llm, confidence 0.7</i>)
Why	The code slice shows a call to fx Settlement Service. settle Leg(currency Pair, amount), which is flagged as reaching an I/O sink. The method name 'settleLeg' strongly suggests mutation (settlement is typically a financial state change). However, the actual implementation of settleLeg is not visible in the provided code slice or call graph—I cannot observe whether it performs idempotent operations (e.g., checks if already settled) or has deduplication logic. The handler reads currencyPair and lockedRate, passes them to the service, and returns a boolean 'settled', but there is no visible idempotency key being constructed or checked in this handler code.
Reads	<code>currencyPair: string</code>
Writes	<code>settled: boolean</code>
Call graph	<code>FxSettlementService.settleLeg</code>

Details — `check-identity-bureau`

What it does	Queries an external identity bureau for applicant status and returns the result or throws a retrieable error if unavailable.
--------------	--

Language / SDK	java · spring-zeebe
Temporal Activity	Check-identity-bureau
Registered as	<code>@JobWorker(type = "check-identity-bureau", autoComplete = false)</code>
Side effect	idempotent (<i>llm, confidence 0.75</i>)
Why	The worker calls Identity Bureau Service. check Bureau(applicant Ref), which reaches an I/O sink. The operation is a query (read-like) against an external system that returns a status. The code throws BUREAU_UNAVAILABLE on transient failures, suggesting retries are safe. However, the exact semantics of checkBureau are not visible: if the bureau internally mutates state on each query (e.g., logging access attempts with side effects), this could be non-idempotent. Given typical identity bureau behavior is read-only lookup, classifying as idempotent is reasonable, but confidence is reduced because the external system's behavior is assumed, not proven from the code slice.
Suggested retry policy	non-retryable: BUREAU_UNAVAILABLE
Writes	<code>bureauStatus: string</code>
Call graph	Identity Bureau Service. check Bureau , Logger.warn
Error mapping	IdentityBureauService thrown in Identity Bureau Service. check Bureau → BUREAU_UNAVAILABLE
Error mapping	BureauUnavailableException thrown in Identity Bureau Service. check Bureau → BUREAU_UNAVAILABLE
newCompleteCommand → Go	<code>return output, nil</code>
newThrowErrorCommand → Go	<code>Non Retryable Application Error(type="BUREAU_UNAVAILABLE")</code>

Details — [lock-fx-rate](#)

What it does	Locks an FX rate for a given currency pair by calling an external settlement service and storing the result.
Language / SDK	java · spring-zeebe
Temporal Activity	Lock-fx-rate
Registered as	<code>@JobWorker(type = "lock-fx-rate", autoComplete = false)</code>
Side effect	idempotent-with-key (<i>llm, confidence 0.65</i>)

Why	The worker calls Fx Settlement Service. <code>lockRate(currencyPair)</code> , which is an I/O sink that likely reserves or records an exchange rate in an external system. The <code>currencyPair</code> variable is read and passed directly to this method. If the external service guarantees that locking the same currency pair multiple times is safe (either returns the same rate or deduplicates by pair), then <code>currencyPair</code> serves as a natural idempotency key. However, the code slice does not reveal whether the <code>FxSettlementService</code> enforces this semantic—rate-locking systems sometimes create distinct locks on each call. The simulated outage (<code>job.getKey() % 5 == 0</code>) suggests retries are expected, supporting idempotent design, but this is circumstantial.
Reads	<code>currencyPair: string</code>
Writes	<code>lockedRate: double</code>
Call graph	<code>Logger.warn</code> , <code>FxSettlementService.lockRate</code>
<code>newCompleteCommand</code> → Go	<code>return output, nil</code>

Details — [reserve-funds](#)

What it does	Reserves funds for an approved loan application in the disbursement ledger
Language / SDK	java · spring-zeebe
Temporal Activity	Reserve-funds
Registered as	<code>@JobWorker(type = "reserve-funds", autoComplete = false)</code>
Side effect	idempotent-with-key (<i>llm, confidence 0.75</i>)
Why	The worker calls Disbursement Service. <code>reserveFunds(applicationId, amount)</code> which mutates external state (ledger reservation). The code explicitly handles 'reservation contention' and retries with backoff when <code>reserved=false</code> , indicating the underlying system may reject duplicate reservations. The <code>applicationId</code> serves as a natural deduplication key for fund reservations—a given application should only have funds reserved once. However, whether the <code>DisbursementService</code> actually enforces idempotency via <code>applicationId</code> (vs. allowing duplicate reservations that must be manually reconciled) is not visible in this code slice.
Suggested idempotency key	<code>applicationId</code>
Reads	<code>applicationId: string</code>
Call graph	<code>DisbursementService.reserveFunds</code> , <code>Logger.warn</code>

newCompleteCommand → Go	<code>return output, nil</code>
newFailCommand → Go	<code>retry via RetryPolicy</code>

Details — [submit-payment-rail](#)

What it does	Submits a payment leg to an external payment rail using the applicationId as the submission identifier.
Language / SDK	java · spring-zeebe
Temporal Activity	Submit-payment-rail
Registered as	<code>@JobWorker(type = "submit-payment-rail", autoComplete = false)</code>
Side effect	idempotent-with-key (<i>ilm, confidence 0.75</i>)
Why	The worker calls Payment Rail Service. <code>submitLeg(applicationId, amount, currencyPair)</code> , which is a side-effecting I/O operation to an external payment system. The applicationId is passed as the first parameter to <code>submitLeg</code> , suggesting it serves as a natural idempotency key for the payment rail to deduplicate submissions. However, whether the external payment rail actually honors applicationId for deduplication cannot be verified from this code slice alone—this depends on the payment rail's implementation.
Suggested idempotency key	<code>applicationId</code>
Suggested retry policy	non-retryable: <code>PAYMENT_RAIL_REJECTED</code>
Reads	<code>applicationId: string</code>
Writes	<code>paymentStatus: string</code>
Call graph	<code>PaymentRailService.submitLeg</code> , <code>Logger.warn</code>
External call	<code>PaymentRailService.submitLeg</code> (service-call)
Error mapping	<code>RuntimeException</code> thrown in <code>PaymentRailService.submitLeg</code> → <code>PAYMENT_RAIL_REJECTED</code>
newCompleteCommand → Go	<code>return output, nil</code>
newThrowErrorCommand → Go	<code>Non Retryable Application Error(type="PAYMENT_ RAIL_ REJECTED")</code>

Details — [arrange-property-access](#)

What it does	Schedules a property access appointment by posting the application ID to an internal scheduling service
Language / SDK	python · pyzeebe

Temporal Activity	ArrangePropertyAccess
Registered as	<code>@worker. task(task_ type="arrange- property- access")</code>
Side effect	idempotent-with-key (<i>llm, confidence 0.75</i>)
Why	The worker calls <code>notification_client.post_json</code> to an external scheduling endpoint with <code>applicationId</code> as payload. The code slice shows a single POST to a scheduling system. If the scheduling service deduplicates by <code>applicationId</code> (common for appointment systems), retries are safe. However, the actual deduplication behavior of the external system is not observable in this code slice, only the fact that <code>applicationId</code> is sent.
Suggested idempotency key	<code>applicationId</code>
Reads	<code>applicationId: string</code>
Writes	<code>propertyAccessArranged: boolean</code>
Call graph	<code>notification_client.post_json</code>
External call	<code>notification_client.post_json</code> (io)

Details — [book-surveyor-slot](#)

What it does	Books a surveyor slot for a loan application by posting the <code>applicationId</code> to an internal scheduling service
Language / SDK	python · pyzeebe
Temporal Activity	BookSurveyorSlot
Registered as	<code>@worker. task(task_ type="book- surveyor- slot")</code>
Side effect	idempotent-with-key (<i>llm, confidence 0.75</i>)
Why	The worker performs a single POST to an external scheduling service with <code>applicationId</code> as the payload. The code slice shows no explicit idempotency token or headers, but the <code>applicationId</code> serves as a natural business key that likely allows the scheduling service to dedupe repeated booking requests for the same application. Classification depends on the external service honoring <code>applicationId</code> -based deduplication, which is not verifiable from the provided code.
Suggested idempotency key	<code>applicationId</code>
Reads	<code>applicationId: string</code>
Writes	<code>surveyorSlotBooked: boolean</code>
Call graph	<code>notification_client.post_json</code>

External call	<code>notification_client.post_json</code> (io)
---------------	---

Details — [build-regulatory-dataset](#)

What it does	Builds a regulatory dataset for a given reporting period and returns completion status.
Language / SDK	java · spring-zeebe
Temporal Activity	Build-regulatory-dataset
Registered as	<code>@JobWorker(type = "build-regulatory-dataset")</code>
Side effect	idempotent (<i>llm, confidence 0.75</i>)
Why	The worker calls Regulatory Reporting Service. build Dataset with reportingPeriod as input and returns a boolean result. Dataset construction for a fixed reporting period is typically idempotent (re-running produces the same output dataset), but the code slice does not reveal whether buildDataset performs upserts, overwrites, or creates duplicates. The method name suggests a deterministic build operation rather than incremental mutation, supporting idempotency. However, without visibility into the service implementation, I cannot confirm the external system's deduplication behavior.
Reads	<code>reportingPeriod: boolean</code>
Writes	<code>datasetBuilt: boolean</code>
Call graph	Regulatory Reporting Service. build Dataset

Details — [core-banking-api-call](#)

What it does	Posts loan application data to core banking system using the applicationId as correlation identifier.
Language / SDK	java · spring-zeebe
Temporal Activity	Core-banking-api-call
Registered as	<code>@JobWorker(type = "core-banking-api-call")</code>
Side effect	idempotent-with-key (<i>llm, confidence 0.75</i>)

Why	The worker calls Credit Bureau Service. post To Core Banking, which is flagged as a side-effecting I/O sink. The code reads 'applicationId' from process variables and passes it to the external call. Since applicationId is a natural business key that uniquely identifies the loan application, repeated posts with the same applicationId should be semantically idempotent if the external system deduplicates by this key. However, the actual deduplication behavior depends on the core banking system's implementation, which is not observable in this code slice.
Suggested idempotency key	<code>applicationId</code>
Reads	<code>applicationId: string</code>
Writes	<code>coreBankingPosted: boolean</code>
Call graph	<code>Credit Bureau Service. post To Core Banking</code>
External call	<code>Credit Bureau Service. post To Core Banking</code> (service-call)

Details — `find-affected-clients`

What it does	Fetches a list of clients affected by a rate change from an internal CRM API and returns them as process variables.
Language / SDK	python · pyzeebe
Temporal Activity	<code>FindAffectedClients</code>
Registered as	<code>@worker. task(task_type="find- affected- clients")</code>
Side effect	read-only (<i>llm, confidence 0.95</i>)
Why	The worker performs a single HTTP GET call (<code>notification_client.get_json</code>) to retrieve data from an external system, then extracts a 'clients' field with a fallback default list. There are no write operations, no mutations to external state, and no engine API calls—only reading and returning data. GET requests are typically idempotent by HTTP semantics, and retries would simply re-fetch the same data. The code slice shows no side effects beyond the read operation.
Writes	<code>affectedClients: unknown</code>
Call graph	<code>notification_client.get_json, data.get</code>
External call	<code>notification_client.get_json</code> (io)

Details — `notify-customer`

What it does	Posts a customer notification to an internal HTTP endpoint using client data.
--------------	---

Language / SDK	python · pyzeebe
Temporal Activity	NotifyCustomer
Registered as	<code>@worker. task(task_ type="notify- customer")</code>
Side effect	idempotent-with-key (<i>llm, confidence 0.65</i>)
Why	The worker calls <code>notification_client.post_json</code> with a single 'client' variable, which serves as the notification payload. The client identifier is a natural idempotency key IF the notification service deduplicates by client (e.g., 'do not notify client X twice about event Y'). However, we cannot observe from this code slice whether the notification endpoint actually implements such deduplication—it may accept duplicate POSTs and send multiple notifications. The code does not set an explicit idempotency token in headers or payload beyond 'client'.
Reads	<code>client: string</code>
Writes	<code>customerNotified: boolean</code>
Call graph	<code>notification_client.post_json</code>
External call	<code>notification_client.post_json</code> (io)

Details — [notify-senior-analyst](#)

What it does	Sends a notification to a senior analyst about a specific application via HTTP POST.
Language / SDK	python · pyzeebe
Temporal Activity	NotifySeniorAnalyst
Registered as	<code>@worker. task(task_ type="notify- senior- analyst")</code>
Side effect	idempotent-with-key (<i>llm, confidence 0.75</i>)
Why	The worker calls <code>notification_client.post_json</code> with <code>applicationId</code> as payload to an internal notification endpoint. The <code>applicationId</code> serves as a natural idempotency key since notifying about the same application multiple times should be semantically safe if the notification system deduplicates by application. The code slice shows a single POST operation with no other side effects. However, classification assumes the notification endpoint handles duplicate POSTs gracefully for the same <code>applicationId</code> .
Suggested idempotency key	<code>applicationId</code>
Reads	<code>applicationId: string</code>
Writes	<code>seniorAnalystNotified: boolean</code>

Call graph	notification_client.post_json
External call	notification_client.post_json (io)

Details — [notify-valuation-appointment](#)

What it does	Sends a valuation appointment notification via HTTP POST to an internal notification service using the applicationId.
Language / SDK	python · pyzeebe
Temporal Activity	NotifyValuationAppointment
Registered as	<code>@worker.task(task_type="notify- valuation- appointment")</code>
Side effect	idempotent-with-key (<i>llm, confidence 0.75</i>)
Why	The worker performs a side-effecting HTTP POST (<code>notification_client.post_json</code>) to an external notification service with <code>applicationId</code> as the payload. The <code>applicationId</code> serves as a natural idempotency key because duplicate notifications for the same application would carry identical <code>applicationId</code> values, allowing the downstream notification service to deduplicate if designed accordingly. However, the actual idempotency behavior depends on whether the receiving service honors <code>applicationId</code> for deduplication.
Suggested idempotency key	applicationId
Reads	<code>applicationId: string</code>
Writes	<code>appointmentNotified: boolean</code>
Call graph	notification_client.post_json
External call	notification_client.post_json (io)

Details — [record-supplementary-evidence](#)

What it does	Records supplementary evidence for an applicant in an identity bureau system
Language / SDK	java · spring-zeebe
Temporal Activity	Record-supplementary-evidence
Registered as	<code>@JobWorker(type = "record- supplementary- evidence")</code>
Side effect	idempotent-with-key (<i>llm, confidence 0.75</i>)

Why	The code slice shows a call to Identity Bureau Service. record Supplementary Evidence with applicantRef and evidenceRef as parameters. The method name 'record' indicates a write operation to an external system (identity bureau). The evidenceRef appears to be a unique identifier for the evidence being recorded, making it a natural candidate for idempotency - recording the same evidenceRef multiple times should be safe. However, the actual deduplication behavior depends on the IdentityBureauService implementation and the external identity bureau system, which are not visible in the provided code.
Reads	<code>applicantRef: unknown, evidenceRef: unknown</code>
Call graph	<code>Identity Bureau Service. record Supplementary Evidence</code>

Details — [release-fund-reservation](#)

What it does	Releases a reserved fund allocation for a loan application in the disbursement system
Language / SDK	java · spring-zeebe
Temporal Activity	Release-fund-reservation
Registered as	<code>@JobWorker(type = "release-fund-reservation")</code>
Side effect	idempotent-with-key (<i>Idm, confidence 0.75</i>)
Why	The worker calls Disbursement Service. release Reservation with applicationId as the sole parameter. Releasing a reservation is typically an idempotent operation when keyed by the reservation identifier—releasing an already-released reservation should be a safe no-op. The applicationId serves as a natural idempotency key since it uniquely identifies which reservation to release. However, the actual idempotency depends on how Disbursement Service. release Reservation is implemented externally.
Suggested idempotency key	<code>applicationId</code>
Reads	<code>applicationId: unknown</code>
Call graph	<code>Disbursement Service. release Reservation</code>

Details — [rescreen-sanctions-pep](#)

What it does	Rescreens an applicant against sanctions and PEP databases by posting applicantRef to an external screening service.
Language / SDK	python · pyzeebe

Temporal Activity	RescreenSanctionsPep
Registered as	<code>@worker.task(task_type="rescreen-sanctions-pep")</code>
Side effect	idempotent-with-key (<i>llm, confidence 0.75</i>)
Why	The worker posts applicantRef to an external sanctions screening endpoint. The applicantRef serves as a natural idempotency key because rescreening the same applicant multiple times should yield the same outcome (sanctions-clear or not). However, the actual idempotency guarantee depends on whether the external service '/v1/rescreen' is itself idempotent for repeated posts with the same applicantRef, which cannot be verified from the provided code.
Reads	<code>applicantRef: string</code>
Writes	<code>sanctionsClear: boolean</code>
Call graph	<code>notification_client.post_json</code>
External call	<code>notification_client.post_json</code> (io)

Details — [screen-sanctions-pep](#)

What it does	Screens an applicant against sanctions and PEP watchlists using an external service and returns a clearance flag.
Language / SDK	java · spring-zeebe
Temporal Activity	Screen-sanctions-pep
Registered as	<code>@JobWorker(type = "screen-sanctions-pep")</code>
Side effect	idempotent (<i>llm, confidence 0.75</i>)
Why	The worker calls SanctionsService.screen with applicantRef and returns a boolean sanctionsClear. The code slice shows a pure read operation (screening) with no writes visible. However, the external SanctionsService.screen behavior is not observable from the provided code—if it performs HTTP calls to third-party watchlist APIs, those are typically idempotent GET operations that can be safely retried. The lack of visible state mutation or transactional writes suggests idempotency, but this assumes the external service does not create audit records or consume rate-limited quotas in a non-idempotent way.
Reads	<code>applicantRef: boolean</code>
Writes	<code>sanctionsClear: boolean</code>
Call graph	<code>SanctionsService.screen</code>

Details — [send-complaint-status-update](#)

What it does	Sends a complaint status update notification to an internal service using the complaint ID
Language / SDK	python · pyzeebe
Temporal Activity	SendComplaintStatusUpdate
Registered as	<code>@worker.task(task_type="send-complaint-status-update")</code>
Side effect	idempotent-with-key (<i>llm, confidence 0.75</i>)
Why	The worker makes a single POST call to an internal notification endpoint with complaintId as the payload. The call graph shows notification_client.post_json is the only delegated operation. If the notification service deduplicates by complaintId (a reasonable assumption for status updates), retries are safe. However, the actual deduplication behavior of the external service is not observable in the provided code.
Suggested idempotency key	<code>complaintId</code>
Reads	<code>complaintId: string</code>
Writes	<code>statusUpdateSent: boolean</code>
Call graph	<code>notification_client.post_json</code>
External call	<code>notification_client.post_json</code> (io)

Details — [send-document-reminder](#)

What it does	Sends a document reminder notification to an external service using the applicationId
Language / SDK	python · pyzeebe
Temporal Activity	SendDocumentReminder
Registered as	<code>@worker.task(task_type="send-document-reminder")</code>
Side effect	idempotent-with-key (<i>llm, confidence 0.75</i>)
Why	The worker calls notification_client.post_json to an external notification service, passing applicationId as payload. This is a side-effecting operation (sending a notification). The applicationId serves as a natural deduplication key IF the downstream notification service deduplicates by applicationId. The code shows no explicit idempotency handling (no transaction ID, no check-before-send), so classification depends on external system behavior.
Suggested idempotency key	<code>applicationId</code>

Reads	<code>applicationId: string</code>
Writes	<code>reminderSent: boolean</code>
Call graph	<code>notification_client.post_json</code>
External call	<code>notification_client.post_json</code> (io)

Details — [chase-document](#)

Language / SDK	java · spring-zeebe
Temporal Activity	Chase-document
Registered as	<code>@JobWorker(type = "chase-document")</code>
Side effect	read-only (<i>rule, confidence 0.85</i>)
Why	call graph fully resolved; reaches no known I/O sink (http/jdbc/kafka/file); writes 1 process variable(s) returned as output
Reads	<code>document: unknown</code>
Writes	<code>documentStatus: unknown</code>
Call graph	<code>Logger.info</code>

Details — [classify-transactions-dmn](#)

Language / SDK	java · spring-zeebe
Temporal Activity	Classify-transactions-dmn
Registered as	<code>@JobWorker(type = "classify-transactions-dmn")</code>
Side effect	read-only (<i>rule, confidence 0.85</i>)
Why	call graph fully resolved; reaches no known I/O sink (http/jdbc/kafka/file); writes 1 process variable(s) returned as output
Writes	<code>transactionsClassified: unknown</code>
Call graph	<code>Logger.info</code>

Details — [confirm-fx-settlement](#)

Language / SDK	java · spring-zeebe
Temporal Activity	Confirm-fx-settlement
Registered as	<code>@JobWorker(type = "confirm-fx-settlement")</code>
Side effect	read-only (<i>rule, confidence 0.85</i>)

Why	call graph fully resolved; reaches no known I/O sink (http/jdbc/kafka/file); writes 1 process variable(s) returned as output
Reads	<code>currencyPair: string</code>
Writes	<code>settlementConfirmed: boolean</code>
Call graph	<code>Fx Settlement Service. confirm Settlement</code>

Details — [escalate-sla-breach](#)

Language / SDK	java · spring-zeebe
Temporal Activity	<code>Escalate-sla-breach</code>
Registered as	<code>@JobWorker(type = "escalate-sla-breach")</code>
Side effect	read-only (<i>rule, confidence 0.85</i>)
Why	call graph fully resolved; reaches no known I/O sink (http/jdbc/kafka/file); writes 1 process variable(s) returned as output
Writes	<code>slaBreachSeverity: unknown</code>
Call graph	<code>Logger.info</code>

Details — [flag-payment-for-review](#)

Language / SDK	java · spring-zeebe
Temporal Activity	<code>Flag-payment-for-review</code>
Registered as	<code>@JobWorker(type = "flag-payment-for-review")</code>
Side effect	read-only (<i>rule, confidence 0.85</i>)
Why	call graph fully resolved; reaches no known I/O sink (http/jdbc/kafka/file); writes 1 process variable(s) returned as output
Reads	<code>applicationId: string</code>
Writes	<code>flaggedForReview: unknown</code>
Call graph	<code>Logger.info</code>

Details — [flag-product-book](#)

Language / SDK	python · pyzeebe
Temporal Activity	<code>FlagProductBook</code>
Registered as	<code>@worker.task(task_type="flag-product-book")</code>

Side effect	read-only (<i>rule, confidence 0.85</i>)
Why	call graph fully resolved; reaches no known I/O sink (http/jdbc/kafka/file); writes 1 process variable(s) returned as output
Writes	<code>productBookFlagged: boolean</code>
Call graph	<code>logger.info</code>

Details — [handle-case-sla-breach](#)

Language / SDK	java · spring-zeebe
Temporal Activity	<code>Handle-case-sla-breach</code>
Registered as	<code>@JobWorker(type = "handle-case-sla-breach")</code>
Side effect	read-only (<i>rule, confidence 0.85</i>)
Why	call graph fully resolved; reaches no known I/O sink (http/jdbc/kafka/file); writes 1 process variable(s) returned as output
Writes	<code>caseSlaBreachHandled: unknown</code>
Call graph	<code>Logger.info</code>

Details — [legacy-fx-reconcile](#)

Language / SDK	java · spring-zeebe
Temporal Activity	<code>Legacy-fx-reconcile</code>
Registered as	<code>@JobWorker(type = "legacy-fx-reconcile")</code>
Side effect	read-only (<i>rule, confidence 0.85</i>)
Why	call graph fully resolved; reaches no known I/O sink (http/jdbc/kafka/file)
Reads	<code>currencyPair: unknown</code>
Call graph	<code>Logger.warn, Fx Settlement Service. reconcile Ledger</code>

Details — [manual-onboarding-review](#)

Language / SDK	java · spring-zeebe
Temporal Activity	<code>Manual-onboarding-review</code>
Registered as	<code>@JobWorker(type = "manual-onboarding-review")</code>
Side effect	read-only (<i>rule, confidence 0.85</i>)

Why	call graph fully resolved; reaches no known I/O sink (http/jdbc/kafka/file); writes 1 process variable(s) returned as output
Writes	<code>manualOnboardingReviewed</code> : unknown
Call graph	<code>Logger.info</code>

Details — [process-cancellation](#)

Language / SDK	java · spring-zeebe
Temporal Activity	Process-cancellation
Registered as	<code>@JobWorker(type = "process-cancellation")</code>
Side effect	read-only (<i>rule, confidence 0.85</i>)
Why	call graph fully resolved; reaches no known I/O sink (http/jdbc/kafka/file); writes 1 process variable(s) returned as output
Writes	<code>cancellationProcessed</code> : unknown
Call graph	<code>Logger.info</code>

Details — [process-withdrawal](#)

Language / SDK	java · spring-zeebe
Temporal Activity	Process-withdrawal
Registered as	<code>@JobWorker(type = "process-withdrawal")</code>
Side effect	read-only (<i>rule, confidence 0.85</i>)
Why	call graph fully resolved; reaches no known I/O sink (http/jdbc/kafka/file); writes 1 process variable(s) returned as output
Writes	<code>withdrawalProcessed</code> : unknown
Call graph	<code>Logger.info</code>

Details — [reconcile-fx-ledger](#)

Language / SDK	java · spring-zeebe
Temporal Activity	Reconcile-fx-ledger
Registered as	<code>@JobWorker(type = "reconcile-fx-ledger")</code>
Side effect	read-only (<i>rule, confidence 0.85</i>)

Why	call graph fully resolved; reaches no known I/O sink (http/jdbc/kafka/file); writes 1 process variable(s) returned as output
Reads	<code>currencyPair: string</code>
Writes	<code>ledgerReconciled: boolean</code>
Call graph	<code>Fx Settlement Service. reconcile Ledger</code>

10. Process & Decision Models (from code)

BPMN Processes (20)

[account-closure](#) — Account Closure · repo [camunda-sample-application](#)

Source: [models/account-closure.bpmn](#)

```
BPMN process "Account Closure" (id: account-closure) - executable.
Contains 7 flow node(s) and 6 sequence flow(s): 2 endEvent, 1 eventBasedGateway, 2 intermediateCatchEvent, 1
receiveTask, 1 startEvent.
Correlates message(s): Message_ClosureConfirmed.

Flow:
  Start: none event "Closure requested"
  Receive task "Await final statement acknowledgement"
  Event-based gateway "Closure route"
  └─[>]
    Catch message event "Client confirms closure" (message "Message_ClosureConfirmed")
    End: none event "Closed (client confirmed)"
  └─[>]
    Catch timer event "Auto-close window" (timeDuration=P14D)
    End: none event "Closed (auto)"
```

[base-rate-change-broadcast](#) — Base Rate Change Broadcast · repo [camunda-sample-application](#)

Source: [models/base-rate-change-broadcast.bpmn](#)

```
BPMN process "Base Rate Change Broadcast" (id: base-rate-change-broadcast) - executable.
Contains 8 flow node(s) and 9 sequence flow(s): 1 endEvent, 2 inclusiveGateway, 1 intermediateThrowEvent, 3
serviceTask, 1 startEvent.
Service job types: flag-product-book.

Flow:
  Start: none event "Rate change confirmed by treasury"
  Inclusive (OR) gateway "Affected product families"
  └─[Mortgages]
    Service task "Flag mortgage book" (job type "flag-product-book")
    Inclusive (OR) gateway "Join"
    └─[>]
      Throw signal event "Broadcast rate change" (signal "Signal_BaseRateChanged")
      End: terminate event "Broadcast complete"
  └─[Personal loans]
    Service task "Flag personal-loan book" (job type "flag-product-book")
    → Join (already described)
  └─[Savings]
    Service task "Flag savings book" (job type "flag-product-book")
```

→ Join (already described)

complaint-handling — Complaint Handling · repo [camunda-sample-application](#)

Source: [models/complaint-handling.bpmn](#)

BPMN process "Complaint Handling" (id: complaint-handling) – executable.
Contains 14 flow node(s) and 11 sequence flow(s): 3 endEvent, 1 exclusiveGateway, 1 intermediateThrowEvent, 1 manualTask, 1 serviceTask, 3 startEvent, 2 subprocess, 1 task, 1 userTask.
Service job types: send-complaint-status-update.
Correlates message(s): Message_ComplaintReceived.

Flow:

Start: message event "Complaint received" (message "Message_ComplaintReceived")
User task "Triage complaint" (groups complaints-team)
manualTask "Send acknowledgement letter"
Exclusive (XOR) gateway "Severity"
└─[Regulatory breach]
 Throw event "Escalate regulatory breach"
 Sub-process "Investigate complaint"
 End: none event "Complaint resolved"
└─[Standard]
 → Investigate complaint (already described)
Start: none event "Investigation started"
task "Gather case facts"
End: none event "Investigation complete"
Start: timer event "Chase timer" (timeCycle=R/P5D)
Service task "Send status update" (job type "send-complaint-status-update")
End: none event "Update sent"

credit-bureau-assessment — Credit Bureau Assessment · repo [camunda-sample-application](#)

Source: [models/credit-bureau-assessment.bpmn](#)

BPMN process "Credit Bureau Assessment" (id: credit-bureau-assessment) – executable.
Contains 14 flow node(s) and 12 sequence flow(s): 1 boundaryEvent, 1 businessRuleTask, 3 endEvent, 1 exclusiveGateway, 1 intermediateCatchEvent, 1 intermediateThrowEvent, 3 serviceTask, 2 startEvent, 1 subprocess.
Service job types: notify-senior-analyst.
Connectors (Camunda-runtime, not job workers): core-banking-api-call
(com.northwindbank.connectors.CoreBankingApi.v1); io.camunda:http-json:1
(io.camunda.connectors.HttpJson.v2).
Invokes DMN decision(s): affordability-band.

Flow:

Start: none event "Assessment requested"
Connector task "Pull credit bureau report" (connector "io.camunda.connectors.HttpJson.v2", task type "io.camunda:http-json:1")
Connector task "Post to Core Banking API" (connector "com.northwindbank.connectors.CoreBankingApi.v1", task type "core-banking-api-call")
Sub-process "Credit risk evaluation"
Exclusive (XOR) gateway "High value?"
└─[High value]
 Throw event "Escalate to senior analyst"
 Catch signal event "Base rate changed" (signal "Signal_BaseRateChanged")
 End: none event "Assessment complete"
└─[Standard]
 → Base rate changed (already described)

```
Start: none event "Evaluation started"
Business-rule task "Evaluate credit risk" → DMN decision "affordability-band"
End: none event "Evaluation complete"

Boundary events (interrupts / error handling):
- event "Senior review flag raised" on "Credit risk evaluation" routes to Notify senior analyst
```

Connectors (Camunda-runtime — map to a generic connector/HTTP activity, not a hand-written worker):

Element	Task type	Connector template
Post to Core Banking API	core-banking-api-call	com.northwindbank.connectors.Core Banking Api. v1
Pull credit bureau report	io.camunda:http-json:1	io.camunda.connectors.Http Json. v2

customer-onboarding-kyc — Customer Onboarding & KYC · repo [camunda-sample-application](#)

Source: [models/customer-onboarding-kyc.bpmn](#)

```
BPMN process "Customer Onboarding & KYC" (id: customer-onboarding-kyc) - executable.
Contains 6 flow node(s) and 5 sequence flow(s): 2 callActivity, 1 endEvent, 1 intermediateCatchEvent, 1
startEvent, 1 userTask.
Service job types: manual-onboarding-review.
Correlates message(s): Message_ApplicantEvidenceReceived.

Flow:
Start: none event "Onboarding requested"
Call activity "Verify each applicant" → process "identity-verification"
Call activity "Sanctions screening" → process "sanctions-screening"
Catch message event "Extra evidence received" (message "Message_ApplicantEvidenceReceived")
User task "Manual onboarding review"
End: none event "Onboarding complete"
```

document-collection — Document Collection · repo [camunda-sample-application](#)

Source: [models/document-collection.bpmn](#)

```
BPMN process "Document Collection" (id: document-collection) - executable.
Contains 13 flow node(s) and 10 sequence flow(s): 2 boundaryEvent, 3 endEvent, 2 intermediateCatchEvent, 1
receiveTask, 3 serviceTask, 1 startEvent, 1 userTask.
Service job types: chase-document, send-document-reminder.
Correlates message(s): Message_DocumentReceived.

Flow:
Start: none event "Collection started"
Service task "Chase document" (job type "chase-document")
Catch message event "Document received" (message "Message_DocumentReceived")
User task "Manual document upload" (groups ops-back-office)
Receive task "Await final confirmation"
Catch timer event "Cooling-off period" (timeDuration=P35D)
End: none event "Collection complete"

Boundary events (interrupts / error handling):
- timer event "Reminder day 3" (timeDuration=P3D) on "Chase document" routes to Send reminder
- timer event "Reminder day 7" (timeDuration=P7D) on "Chase document" routes to Send final reminder
```

fund-disbursement — Fund Disbursement · repo `camunda-sample-application`

Source: `models/fund-disbursement.bpmn`

BPMN process "Fund Disbursement" (id: fund-disbursement) – executable.
Contains 12 flow node(s) and 9 sequence flow(s): 1 boundaryEvent, 1 callActivity, 2 endEvent, 1 exclusiveGateway, 2 intermediateThrowEvent, 1 sendTask, 3 serviceTask, 1 startEvent.
Service job types: manual-ledger-adjustment, notify-ledger-posted, publish-ledger-posted-message, release-fund-reservation, reserve-funds.
Correlates message(s): Message_LedgerPosted.

Flow:

Start: none event "Disbursement requested"
Service task "Reserve funds on ledger" (job type "reserve-funds")
Service task "Manual ledger adjustment" (job type "manual-ledger-adjustment")
Call activity "Instruct payment" → process "payment-instruction"
Exclusive (XOR) gateway "Payment succeeded?"
└─[Settled]
 Service task "Notify ledger posted" (job type "notify-ledger-posted")
 Throw message event "Ledger posted event" (message "Message_LedgerPosted")
 End: none event "Disbursement settled"
└─[Failed]
 Throw event "Trigger compensation"
 End: none event "Disbursement failed"

Boundary events (interrupts / error handling):

- event "Compensate reservation" on "Reserve funds on ledger"

fx-settlement — FX Settlement · repo `camunda-sample-application`

Source: `models/fx-settlement-v2.bpmn`

BPMN process "FX Settlement" (id: fx-settlement) – executable.
Contains 5 flow node(s) and 4 sequence flow(s): 1 endEvent, 3 serviceTask, 1 startEvent.
Service job types: confirm-fx-settlement, lock-fx-rate, settle-fx-leg.

Flow:

Start: none event "Settlement requested"
Service task "Lock FX rate" (job type "lock-fx-rate")
Service task "Settle currency leg" (job type "settle-fx-leg")
Service task "Confirm settlement" (job type "confirm-fx-settlement")
End: none event "Settlement complete"

fx-settlement — FX Settlement · repo `camunda-sample-application`

Source: `models/fx-settlement-v3.bpmn`

BPMN process "FX Settlement" (id: fx-settlement) – executable.
Contains 6 flow node(s) and 5 sequence flow(s): 1 endEvent, 4 serviceTask, 1 startEvent.
Service job types: confirm-fx-settlement, lock-fx-rate, reconcile-fx-ledger, settle-fx-leg.

Flow:

Start: none event "Settlement requested"
Service task "Lock FX rate" (job type "lock-fx-rate")
Service task "Settle currency leg" (job type "settle-fx-leg")
Service task "Confirm settlement" (job type "confirm-fx-settlement")


```
Service task "Reconcile against ledger" (job type "reconcile-fx-ledger")
End: none event "Settlement complete"
```

fx-settlement — FX Settlement · repo [camunda-sample-application](#)

Source: [models/fx-settlement.bpmn](#)

```
BPMN process "FX Settlement" (id: fx-settlement) – executable.
Contains 4 flow node(s) and 3 sequence flow(s): 1 endEvent, 2 serviceTask, 1 startEvent.
Service job types: lock-fx-rate, settle-fx-leg.
```

Flow:

```
Start: none event "Settlement requested"
Service task "Lock FX rate" (job type "lock-fx-rate")
Service task "Settle currency leg" (job type "settle-fx-leg")
End: none event "Settlement complete"
```

identity-verification — Identity Verification · repo [camunda-sample-application](#)

Source: [models/identity-verification.bpmn](#)

```
BPMN process "Identity Verification" (id: identity-verification) – executable.
Contains 10 flow node(s) and 9 sequence flow(s): 2 boundaryEvent, 1 callActivity, 2 endEvent, 1
exclusiveGateway, 2 serviceTask, 1 startEvent, 1 userTask.
Service job types: check-identity-bureau, record-supplementary-evidence.
Correlates message(s): Message_SupplementaryEvidence.
```

Flow:

```
Start: none event "Verification requested"
Service task "Check identity bureau" (job type "check-identity-bureau")
Exclusive (XOR) gateway "Verified?"
├─[Verified]
│   Call activity "Collect ID documents" → process "document-collection"
│   End: none event "Verification complete"
└─[Unverified]
    User task "Manual identity review" (groups identity-ops)
    → Collect ID documents (already described)
```

Boundary events (interrupts / error handling):

- error event "Bureau unverifiable" (error "BUREAU_UNAVAILABLE") on "Check identity bureau" routes to Manual identity review
- message event "Extra evidence arrives" (message "Message_SupplementaryEvidence") on "Check identity bureau" routes to Record supplementary evidence

loan-application — Loan Application · repo [camunda-sample-application](#)

Source: [models/loan-application.bpmn](#)

```
BPMN process "Loan Application" (id: loan-application) – executable.
Contains 18 flow node(s) and 16 sequence flow(s): 1 boundaryEvent, 5 callActivity, 4 endEvent, 1
exclusiveGateway, 2 parallelGateway, 2 serviceTask, 2 startEvent, 1 subprocess.
Service job types: process-cancellation, process-withdrawal.
Correlates message(s): Message_ApplicationCancelled, Message-WithdrawalRequested.
```

Flow:

```
Start: none event "Application submitted"
Parallel (AND) gateway "Split onboarding / credit"
```

```

    └─[→]
      Call activity "Onboard & verify KYC" → process "customer-onboarding-kyc"
      Parallel (AND) gateway "Join onboarding / credit"
      └─[→]
        Call activity "Collect documents" → process "document-collection"
        Call activity "Underwriting review" → process "=underwritingProcessId"
        Exclusive (XOR) gateway "Decision"
        └─[Approved]
          Call activity "Disburse funds" → process "fund-disbursement"
          End: none event "Funded"
        └─[Referred]
          → Underwriting review (already described)
        └─[default]
          End: terminate event "Declined"

    └─[→]
      Call activity "Assess credit bureau" → process "credit-bureau-assessment"
      → Join onboarding / credit (already described)
      Start: message event "Withdrawal requested" (message "Message-WithdrawalRequested")
      Service task "Process withdrawal" (job type "process-withdrawal")
      End: terminate event "Withdrawn"

```

Boundary events (interrupts / error handling):

- message event "Client cancels" (message "Message_ApplicationCancelled") on "Underwriting review" routes to Process cancellation

payment-instruction — Payment Instruction · repo [camunda-sample-application](#)

Source: [models/payment-instruction.bpmn](#)

BPMN process "Payment Instruction" (id: payment-instruction) – executable.
 Contains 15 flow node(s) and 10 sequence flow(s): 4 endEvent, 1 exclusiveGateway, 1 intermediateCatchEvent, 1 intermediateThrowEvent, 3 serviceTask, 3 startEvent, 2 subprocess.
 Service job types: flag-payment-for-review, retry-alternate-rail, submit-payment-rail.

Flow:

```

Start: none event "Instruction requested"
Sub-process "Execute payment leg"
Throw event "To result handling"
Start: none event "Leg started"
Service task "Submit to payment rail" (job type "submit-payment-rail")
End: none event "Leg settled"
Start: error event "Rail error caught" (error "PAYMENT_RAIL_REJECTED")
Service task "Retry via alternate rail" (job type "retry-alternate-rail")
End: error event "Error end" (error "INSUFFICIENT_FUNDS")

```

property-valuation-scheduling — Property Valuation Scheduling · repo [camunda-sample-application](#)

Source: [models/property-valuation-scheduling.bpmn](#)

BPMN process "Property Valuation Scheduling" (id: property-valuation-scheduling) – executable.
 Contains 3 flow node(s) and 3 sequence flow(s): 1 callActivity, 1 endEvent, 1 startEvent.

Flow:

```

Start: timer event "Nightly scheduling run" (timeCycle=R/PT24H)
ah_schedule_visit

```

rate-change-notification — Rate Change Notification · repo [camunda-sample-application](#)

Source: `models/rate-change-notification.bpmn`

BPMN process "Rate Change Notification" (id: rate-change-notification) – executable.
Contains 4 flow node(s) and 3 sequence flow(s): 1 endEvent, 2 serviceTask, 1 startEvent.
Service job types: find-affected-clients, notify-customer.

Flow:

Start: signal event "Rate change broadcast received" (signal "Signal_BaseRateChanged")
Service task "Find affected clients" (job type "find-affected-clients")
Service task "Notify client" (job type "notify-customer")
End: none event "Notifications sent"

regulatory-reporting-batch — Regulatory Reporting Batch · repo `camunda-sample-application`

Source: `models/regulatory-reporting-batch.bpmn`

BPMN process "Regulatory Reporting Batch" (id: regulatory-reporting-batch) – executable.
Contains 6 flow node(s) and 5 sequence flow(s): 1 businessRuleTask, 1 callActivity, 1 endEvent, 2 scriptTask, 1 startEvent.
Service job types: build-regulatory-dataset, classify-transactions-dmn.

Flow:

Start: timer event "Nightly batch trigger" (timeCycle=30 17 3 * * ?)
Script task "Compute reporting period"
Script task "Build regulatory dataset"
Business-rule task "Classify transactions"
Call activity "Submit per currency pair" → process "fx-settlement"
End: none event "Batch complete"

sanctions-screening — Sanctions Screening · repo `camunda-sample-application`

Source: `models/sanctions-screening.bpmn`

BPMN process "Sanctions Screening" (id: sanctions-screening) – executable.
Contains 6 flow node(s) and 5 sequence flow(s): 1 endEvent, 1 intermediateCatchEvent, 2 serviceTask, 2 startEvent.
Service job types: rescreen-sanctions-pep, screen-sanctions-pep.

Flow:

Start: none event "Screening requested"
Service task "Screen against sanctions/PEP lists" (job type "screen-sanctions-pep")
End: none event "Screening complete"
Start: timer event "Every 6 hours" (timeCycle=R/PT6H)
Service task "Re-screen applicant" (job type "rescreen-sanctions-pep")
→ Screening complete (already described)

two-pool-spike-analyst-pool — Analyst Pool (non-executable) · repo `camunda-sample-application`

Source: `models/two-pool-spike.bpmn`

BPMN process "Analyst Pool (non-executable)" (id: two-pool-spike-analyst-pool) – non-executable (referenced only).
Contains 5 flow node(s) and 4 sequence flow(s): 1 complexGateway, 1 endEvent, 1 intermediateCatchEvent, 1 serviceTask, 1 startEvent.
Service job types: branch-na.

Flow:

```
Start: none event "Start"
complexGateway "Complex routing"
└─[→]
  Service task "Branch" (job type "branch-na")
  Catch conditional event "Balance threshold reached"
End: none event "End"
```

two-pool-spike-executable — Two-Pool Spike (Executable) · repo [camunda-sample-application](#)

Source: [models/two-pool-spike.bpmn](#)

BPMN process "Two-Pool Spike (Executable)" (id: two-pool-spike-executable) – executable.
Contains 3 flow node(s) and 2 sequence flow(s): 1 endEvent, 1 serviceTask, 1 startEvent.
Service job types: spike-do-work.

Flow:

```
Start: none event "Start"
Service task "Do work" (job type "spike-do-work")
End: none event "End"
```

underwriting-review — Underwriting Review · repo [camunda-sample-application](#)

Source: [models/underwriting-review.bpmn](#)

BPMN process "Underwriting Review" (id: underwriting-review) – executable.
Contains 13 flow node(s) and 12 sequence flow(s): 1 boundaryEvent, 3 endEvent, 1 eventBasedGateway, 1 exclusiveGateway, 2 intermediateCatchEvent, 2 serviceTask, 1 startEvent, 2 userTask.
Service job types: escalate-sla-breach, handle-case-sla-breach.
Correlates message(s): Message_ClientResponded.

Flow:

```
Start: none event "Case ready for review"
User task "Underwriter review" (assignee =caseOwnerUserId)
Exclusive (XOR) gateway "Needs second sign-off?"
└─[Over threshold]
  User task "Second underwriter sign-off" (groups senior-underwriters)
  Event-based gateway "Wait for outcome"
  └─[→]
    Catch message event "Client responds" (message "Message_ClientResponded")
    End: none event "Decision recorded"
  └─[→]
    Catch timer event "SLA window" (timeDuration=PT48H)
    Service task "Escalate SLA breach" (job type "escalate-sla-breach")
    End: none event "SLA breached"
└─[Under threshold]
  → Wait for outcome (already described)
```

Boundary events (interrupts / error handling):

- timer event "Case SLA expired" (timeDuration=P5D) on "Underwriter review" routes to Handle case SLA breach

DMN Decisions (5)

affordability-band — Affordability Band · repo [camunda-sample-application](#)

hit policy **UNIQUE**, 3 rule(s).

Source: [models/credit-risk.dmn](#)

#	in: Monthly Income	in: Existing Debt Ratio	out: Affordability Band
1	<code>>= 5000</code>	<code>< 0.3</code>	"HIGH"
2	<code>[2000..5000)</code>	<code>< 0.5</code>	"MEDIUM"
3	-	-	"LOW"

[override-precedence](#) — **Override Precedence** · repo [camunda-sample-application](#)

hit policy **PRIORITY**, 3 rule(s), depends on [risk-flags](#).

Source: [models/credit-risk.dmn](#)

#	in: Manual Override Requested	in: Has Sanctions Flag	out: Final Decision
1	-	<code>true</code>	"DECLINE"
2	<code>true</code>	<code>false</code>	"MANUAL_REVIEW"
3	<code>false</code>	<code>false</code>	"PROCEED"

[risk-flags](#) — **Risk Flags** · repo [camunda-sample-application](#)

hit policy **COLLECT**, 3 rule(s), depends on [affordability-band](#).

Source: [models/credit-risk.dmn](#)

#	in: Affordability Band	in: County Court Judgment Count	in: Sanctions Hit	out: Risk Flag
1	"HIGH"	-	-	"AFFORDABILITY_HIGH_RISK"
2	-	<code>>= 1</code>	-	"CCJ_ON_FILE"
3	-	-	<code>true</code>	"SANCTIONS_HIT"

[fee-waiver](#) — **Fee Waiver** · repo [camunda-sample-application](#)

hit policy **literal**, 0 rule(s).

Source: [models/fee-waiver.dmn](#)

No decision-table rules extracted (literal expression or empty table).

[pricing-tier](#) — **Pricing Tier** · repo [camunda-sample-application](#)

hit policy **FIRST**, 3 rule(s).

Source: [models/pricing-tier.dmn](#)

#	in: Affordability Band	in: Requested Amount	out: Interest Rate Tier
1	"HIGH"	<code>> 100000</code>	"PREMIUM"
2	"HIGH"	-	"STANDARD_PLUS"
3	-	-	"STANDARD"

11. Entry Points

13 production entry point(s).

Kind	Command	Starts Process	Origin	Location
code	create Process Instance	credit- bureau- assessment	main	seed/reseed_fixes.py:43
code	create Process Instance	payment-instruction	main	seed/reseed_fixes.py:52
code	create Process Instance	fx-settlement	main	seed/reseed_fixes.py:63
code	create Process Instance	—	main	seed/run_seed.py:111
code	create Process Instance	—	main	seed/run_seed.py:129
message	broadcastSignal	BaseRateChanged	main	seed/run_seed.py:146
message	broadcastSignal	SanctionsListUpdated	main	seed/run_seed.py:153
message	publishMessage	ComplaintReceived	main	seed/run_seed.py:161
message	publishMessage	DocumentReceived	main	seed/run_seed.py:177
message	publishMessage	CollectionConfirmed	main	seed/run_seed.py:178
message	publishMessage	ClientResponded	main	seed/run_seed.py:189
code	create Process Instance	loan-application	main	seed/run_seed.py:232
message	publishMessage	—	main	seed/run_seed.py:265

12. Reconciliation

BPMN Job Types vs Runtime Activity:

- Declared but never observed running (16): [arrange-property-access](#), [book-surveyor-slot](#), [core-banking-api-call](#), [escalate-sla-breach](#), [flag-payment-for-review](#), [handle-case-sla-breach](#), [notify-ledger-posted](#), [notify-senior-analyst](#), [notify-valuation-appointment](#), [process-cancellation](#), [process-withdrawal](#), [record-supplementary-evidence](#), [release-fund-reservation](#), [retry-alternate-rail](#), [send-complaint-status-update](#), [send-document-reminder](#) — a candidate dead path; confirm before dropping.

BPMN Job Types vs Workers:

Category	Count	Items
----------	-------	-------

BPMN job types	33	arrange-property-access , book-surveyor-slot , build-regulatory-dataset , chase-document , check-identity-bureau , classify-transactions-dmn , confirm-fx-settlement , core-banking-api-call , escalate-sla-breach , find-affected-clients , flag-payment-for-review , flag-product-book , ... (+21 more)
Matched	31	arrange-property-access , book-surveyor-slot , build-regulatory-dataset , chase-document , check-identity-bureau , classify-transactions-dmn , confirm-fx-settlement , core-banking-api-call , escalate-sla-breach , find-affected-clients , flag-payment-for-review , flag-product-book , ... (+19 more)
BPMN-only (no worker)	2	io.camunda:http-json:1 , manual-ledger-adjustment
Worker-only (no BPMN)	3	legacy-fx-reconcile , manual-onboarding-review , publish-ledger-posted-message

Worker-only job types have code but no matching task in the scanned BPMN inventory — they belong to processes not present in the cluster scan (a different/parallel implementation or an un-scanned deployment). Confirm whether they are in migration scope.

BPMN Processes (cluster inventory vs code):

Category	Count	Items
In cluster inventory	16	account-closure , base-rate-change-broadcast , complaint-handling , credit-bureau-assessment , customer-onboarding-kyc , document-collection , fund-disbursement , fx-settlement , identity-verification , loan-application , payment-instruction , property-valuation-scheduling , ... (+4 more)
Parsed from code	18	account-closure , base-rate-change-broadcast , complaint-handling , credit-bureau-assessment , customer-onboarding-kyc , document-collection , fund-disbursement , fx-settlement , identity-verification , loan-application , payment-instruction , property-valuation-scheduling , ... (+6 more)
Matched	16	account-closure , base-rate-change-broadcast , complaint-handling , credit-bureau-assessment , customer-onboarding-kyc , document-collection , fund-disbursement , fx-settlement , identity-verification , loan-application , payment-instruction , property-valuation-scheduling , ... (+4 more)
Code-only (BPMN file not in cluster scan)	2	two-pool-spike-analyst-pool , two-pool-spike-executable

DMN Decisions (cluster inventory vs code):

Category	Count	Items
In cluster inventory	1	affordability-band
Parsed from code	5	affordability-band , fee-waiver , override-precedence , pricing-tier , risk-flags
Matched	1	affordability-band

Code-only (DMN not in cluster scan)	4	fee-waiver , override-precedence , pricing-tier , risk-flags
-------------------------------------	---	--

User tasks in BPMN: 6

Connector job types: [core-banking-api-call](#), [io.camunda:http-json:1](#)

13. Gaps & Coverage

Severity	Code	Message
info	no-test-oracles	no input→output test oracles were extracted; generated code has no dual-run seed from existing tests

14. Security

Credentials hardcoded in the application source that must be moved to a secret store before or during migration. Dev/deploy config ([.env](#), shell scripts) is scanned but excluded here as out-of-scope.

Severity	Kind	Field	Location
major	hardcoded-credential	TOKEN	workers- java/ src/ main/ java/ com/ northwindbank/ clients/ Core Banking Client. java: 24
major	hardcoded-credential	API_KEY	workers- python/ northwind_ workers/ notification_ client. py: 18

15. Work Item Register

Every discrete piece of work this migration requires, grouped by the kind of work rather than by the process it lives in. Item IDs are stable across assessments, so this can be re-imported after a re-scan without duplicating rows.

85 work items — 5 Low · 70 Medium · 10 High

Work type	Low	Medium	High	Total
Redesign	0	9	5	14
Data remediation	0	10	0	10
Human task handling	0	6	0	6
Decision logic	0	1	0	1
Connector replacement	0	2	0	2
Workflow translation	5	9	2	16
Activity implementation	0	33	0	33

Platform dependency (customer-side)	0	0	3	3
Total	5	70	10	85

Redesign (14)

Constructs with no mechanical translation — a developer decides the Temporal shape.

ID	Location	What to do	Why	Complexity	Size	Blocked by
<code>redesign: fund-disbursement : svc_release_reservation</code>	<code>fund-disbursement</code> · Release fund reservation	Translate with a design decision — the generator will scaffold this and flag it for completion.	compensation. handler has no direct Temporal equivalent	Medium	21	—
<code>redesign: base-rate-change-broadcast: throw_broadcast</code>	<code>base-rate-change-broadcast</code> · Broadcast rate change	Redesign this construct against Temporal primitives — no mechanical translation exists.	No native equivalent	High	20	—
<code>redesign: fund-disbursement : boundary_compensate_reservation</code>	<code>fund-disbursement</code> · Compensate reservation	Translate with a design decision — the generator will scaffold this and flag it for completion.	compensation. boundary has no direct Temporal equivalent	Medium	20	—
<code>redesign: fund-disbursement : throw_compensate</code>	<code>fund-disbursement</code> · Trigger compensation	Translate with a design decision — the generator will scaffold this and flag it for completion.	Ordering differs	Medium	20	—
<code>redesign: identity-verification : irreducible</code>	<code>identity-verification</code>	Decompose this process into composable workflows, or emit it as a labelled state machine.	The control-flow graph is irreducible — it cannot be expressed as nested Go blocks.	High	20	—
<code>redesign: property-valuation-scheduling: ah_schedule_visit</code>	<code>property-valuation-scheduling</code> · Arrange surveyor visit	Redesign this construct against Temporal primitives — no mechanical translation exists.	adHocSubprocess has no direct Temporal equivalent	High	20	—

redesign: rate- change- notification : start	rate- change- notification · Rate change broadcast received	Redesign this construct against Temporal primitives — no mechanical translation exists.	signal.start has no direct Temporal equivalent	High	20	—
redesign: underwriting - review: irreducible	underwriting- review	Decompose this process into composable workflows, or emit it as a labelled state machine.	The control-flow graph is irreducible — it cannot be expressed as nested Go blocks.	High	20	—
redesign: complaint- handling: event_ status_ chaser	complaint- handling · Status chaser	Translate with a design decision — the generator will scaffold this and flag it for completion.	subprocess. event Non Interrupting has no direct Temporal equivalent	Medium	8	—
redesign: credit- bureau- assessment: boundary_ senior_ flag	credit- bureau- assessment · Senior review flag raised	Translate with a design decision — the generator will scaffold this and flag it for completion.	escalation. non Interrupting has no direct Temporal equivalent	Medium	8	—
redesign: credit- bureau- assessment: br_ credit_ risk	credit- bureau- assessment · Evaluate credit risk	Translate with a design decision — the generator will scaffold this and flag it for completion.	DMN strategy decision	Medium	8	—
redesign: customer- onboarding- kyc: catch_ extra_ evidence	customer- onboarding- kyc · Extra evidence received	Translate with a design decision — the generator will scaffold this and flag it for completion.	Doc 02 splits this row AUTO / ASSISTED by correlation-key complexity — doc 03 §5.8's simple- variable- vs- computed classification (from the census) is the split key.	Medium	8	—
redesign: loan- application: event_ withdrawal	loan- application · Client withdraws	Translate with a design decision — the generator will scaffold this and flag it for completion.	subprocess. event Interrupting has no direct Temporal equivalent	Medium	8	—

redesign: payment- instruction: sub_error_ handling	payment- instruction · Handle rail error	Translate with a design decision — the generator will scaffold this and flag it for completion.	subprocess. event Interrupting has no direct Temporal equivalent	Medium	8	—
---	---	--	---	--------	---	---

Data remediation (10)

Variable-handling findings to verify before the process can be trusted.

ID	Location	What to do	Why	Complexity	Size	Blocked by
data- remediation: customer- onboarding- kyc: 1	customer- onboarding- kyc	Verify and correct the variable handling before porting this process.	Call activity "call_ verify_per_ applicant" → "identity- verification": approximate true input set is applicantRef, requiredDocuments — cross-check against this call site's own ioMapping before porting	Medium	<i>not sized</i>	—
data- remediation: customer- onboarding- kyc: 2	customer- onboarding- kyc	Verify and correct the variable handling before porting this process.	Element "call_ verify_per_ applicant" input mapping to "applicantRef" is a non-trivial FEEL expression (=applicant.ref) — translation risk, see doc 07	Medium	<i>not sized</i>	—
data- remediation: identity- verification : 1	identity- verification	Verify and correct the variable handling before porting this process.	Call activity "call_documents" → "document- collection": approximate true input set is applicationId, requiredDocuments — cross-check against this call site's own ioMapping before porting	Medium	<i>not sized</i>	—

data-remediation: loan-application: 1	loan-application	Verify and correct the variable handling before porting this process.	Call activity "call_disbursement" → "fund-disbursement": approximate true input set is applicationId, approvedAmount — cross-check against this call site's own ioMapping before porting	Medium	not sized	—
data-remediation: loan-application: 2	loan-application	Verify and correct the variable handling before porting this process.	Call activity "call_documents" → "document-collection": approximate true input set is applicationId, requiredDocuments — cross-check against this call site's own ioMapping before porting	Medium	not sized	—
data-remediation: loan-application: 3	loan-application	Verify and correct the variable handling before porting this process.	Call activity "call_onboarding" → "customer-onboarding- kyc": approximate true input set is applicants, applicationId — cross-check against this call site's own ioMapping before porting	Medium	not sized	—
data-remediation: loan-application: 4	loan-application	Verify and correct the variable handling before porting this process.	Call activity "call_underwriting" → "=underwriting Process Id": true child input set could not be resolved (calledProcessId is a FEEL expression (dynamic dispatch) — cannot statically resolve the child) — verify the variables it needs by hand	Medium	not sized	—

data-remediation: property-valuation-scheduling: 1	property-valuation-scheduling	Verify and correct the variable handling before porting this process.	Call activity "call_documents" → "document-collection": approximate true input set is applicationId, requiredDocuments — cross-check against this call site's own ioMapping before porting	Medium	not sized	—
data-remediation: rate-change-notification: 1	rate-change-notification	Verify and correct the variable handling before porting this process.	Element "mi_notify" is a parallel multi-instance with no output mapping — Camunda's documented race condition here is silently removed on a mechanical port	Medium	not sized	—
data-remediation: regulatory-reporting-batch: 1	regulatory-reporting-batch	Verify and correct the variable handling before porting this process.	Element "mi_submit_per_currency" is a parallel multi-instance with no output mapping — Camunda's documented race condition here is silently removed on a mechanical port	Medium	not sized	—

Human task handling (6)

Signal and update handlers for each human step.

ID	Location	What to do	Why	Complexity	Size	Blocked by
human-task-handling: complaint-handling: complaint-triage-form	complaint-handling · complaint-triage-form	Implement the workflow-side signal/update handlers for this human step, and bind the form to them.	Temporal has no user task or form runtime; the interaction has to be modelled explicitly.	Medium	incl. <i>elsew here</i>	User task service (Tasklist replacement)

human- task- handling: customer- onboarding- kyc: ut_ manual_ review	customer- onboarding- kyc · ut_ manual_ review	Implement the workflow-side signal/update handlers for this human step, and bind the form to them.	Temporal has no user task or form runtime; the interaction has to be modelled explicitly.	Medium	<i>incl. elsew here</i>	User task service (Tasklist replacement)
human- task- handling: document- collection: document- upload- form	document- collection · document- upload- form	Implement the workflow-side signal/update handlers for this human step, and bind the form to them.	Temporal has no user task or form runtime; the interaction has to be modelled explicitly.	Medium	<i>incl. elsew here</i>	User task service (Tasklist replacement)
human- task- handling: identity- verification : identity- manual- review- form	identity- verification · identity- manual- review- form	Implement the workflow-side signal/update handlers for this human step, and bind the form to them.	Temporal has no user task or form runtime; the interaction has to be modelled explicitly.	Medium	<i>incl. elsew here</i>	User task service (Tasklist replacement)
human- task- handling: underwriting - review: underwriting - first- review- form	underwriting- review · underwriting- first- review- form	Implement the workflow-side signal/update handlers for this human step, and bind the form to them.	Temporal has no user task or form runtime; the interaction has to be modelled explicitly.	Medium	<i>incl. elsew here</i>	User task service (Tasklist replacement)
human- task- handling: underwriting - review: underwriting - second- review- form	underwriting- review · underwriting- second- review- form	Implement the workflow-side signal/update handlers for this human step, and bind the form to them.	Temporal has no user task or form runtime; the interaction has to be modelled explicitly.	Medium	<i>incl. elsew here</i>	User task service (Tasklist replacement)

Decision logic (1)

DMN decisions reimplemented as Activities or delegated to a decision service.

ID	Location	What to do	Why	Complexity	Size	Blocked by
decision- logic: credit- bureau- assessment: affordabilit y- band	credit- bureau- assessment · affordability - band	Reimplement this decision as a Temporal Activity, or call it out to a decision service.	Temporal ships no DMN engine.	Medium	<i>incl. elsew here</i>	Decision service (DMN engine replacement)

Connector replacement (2)

Each connector rewritten as a Temporal Activity.

ID	Location	What to do	Why	Complexity	Size	Blocked by
<code>connector-replacement: credit-bureau-assessment: core-banking-api-call</code>	<code>credit-bureau-assessment · core-banking-api-call</code>	Replace this connector with a hand-written Temporal Activity.	Camunda connectors have no Temporal equivalent; the integration has to be written out.	Medium	<i>incl. elsewhere</i>	Connector Activity library
<code>connector-replacement: credit-bureau-assessment: io-camunda-http-json-1</code>	<code>credit-bureau-assessment · io-camunda-http-json-1</code>	Replace this connector with a hand-written Temporal Activity.	Camunda connectors have no Temporal equivalent; the integration has to be written out.	Medium	<i>incl. elsewhere</i>	Connector Activity library

Workflow translation (16)

The mechanically translatable bulk — generated, then reviewed.

ID	Location	What to do	Why	Complexity	Size	Blocked by
<code>workflow-translation: loan-application</code>	<code>loan-application</code>	Generate and review the Temporal workflow for this process.	Grade D process, 18 elements — the mechanically translatable remainder.	High	47	—
<code>workflow-translation: document-collection</code>	<code>document-collection</code>	Generate and review the Temporal workflow for this process.	Grade C process, 14 elements — the mechanically translatable remainder.	Medium	35	—
<code>workflow-translation: underwriting-review</code>	<code>underwriting-review</code>	Generate and review the Temporal workflow for this process.	Grade C process, 13 elements — the mechanically translatable remainder.	Medium	29	—
<code>workflow-translation: regulatory-reporting-batch</code>	<code>regulatory-reporting-batch</code>	Generate and review the Temporal workflow for this process.	Grade C process, 7 elements — the mechanically translatable remainder.	Medium	27	—

workflow-translation: identity-verification	identity-verification	Generate and review the Temporal workflow for this process.	Grade C process, 10 elements — the mechanically translatable remainder.	Medium	25	—
workflow-translation: customer-onboarding-kyc	customer-onboarding-kyc	Generate and review the Temporal workflow for this process.	Grade C process, 7 elements — the mechanically translatable remainder.	Medium	22	—
workflow-translation: property-valuation-scheduling	property-valuation-scheduling	Generate and review the Temporal workflow for this process.	Grade C process, 7 elements — the mechanically translatable remainder.	Medium	21	—
workflow-translation: base-rate-change-broadcast	base-rate-change-broadcast	Generate and review the Temporal workflow for this process.	Grade C process, 8 elements — the mechanically translatable remainder.	Medium	17	—
workflow-translation: complaint-handling	complaint-handling	Generate and review the Temporal workflow for this process.	Grade B process, 14 elements — the mechanically translatable remainder.	Low	14	—
workflow-translation: sanctions-screening	sanctions-screening	Generate and review the Temporal workflow for this process.	Grade B process, 6 elements — the mechanically translatable remainder.	Low	11	—
workflow-translation: account-closure	account-closure	Generate and review the Temporal workflow for this process.	Grade A process, 7 elements — the mechanically translatable remainder.	Low	10	—
workflow-translation: credit-bureau-assessment	credit-bureau-assessment	Generate and review the Temporal workflow for this process.	Grade C process, 14 elements — the mechanically translatable remainder.	Medium	10	—
workflow-translation: rate-change-notification	rate-change-notification	Generate and review the Temporal workflow for this process.	Grade C process, 5 elements — the mechanically translatable remainder.	Medium	10	—

workflow-translation: fund-disbursement	fund-disbursement	Generate and review the Temporal workflow for this process.	Grade D process, 13 elements — the mechanically translatable remainder.	High	8	—
workflow-translation: fx-settlement	fx-settlement	Generate and review the Temporal workflow for this process.	Grade A process, 6 elements — the mechanically translatable remainder.	Low	6	—
workflow-translation: payment-instruction	payment-instruction	Generate and review the Temporal workflow for this process.	Grade B process, 15 elements — the mechanically translatable remainder.	Low	5	—

Activity implementation (33)

One Activity per job type, with retry policy and timeouts.

ID	Location	What to do	Why	Complexity	Size	Blocked by
activity-implementation: arrange-property-access	arrange-property-access	Implement the Temporal Activity for this job type and set its retry policy and timeouts.	Job worker becomes a Temporal Activity, with retries expressed as a RetryPolicy. Side-effect class not yet annotated — confirm whether this job is safe to retry.	Medium	<i>incl. elsewhere</i>	—
activity-implementation: book-surveyor-slot	book-surveyor-slot	Implement the Temporal Activity for this job type and set its retry policy and timeouts.	Job worker becomes a Temporal Activity, with retries expressed as a RetryPolicy. Side-effect class not yet annotated — confirm whether this job is safe to retry.	Medium	<i>incl. elsewhere</i>	—

activity-implementation: build-regulatory-dataset	build-regulatory-dataset	Implement the Temporal Activity for this job type and set its retry policy and timeouts.	Job worker becomes a Temporal Activity, with retries expressed as a RetryPolicy. Side-effect class not yet annotated — confirm whether this job is safe to retry.	Medium	<i>incl. elsewhere</i>	—
activity-implementation: chase-document	chase-document	Implement the Temporal Activity for this job type and set its retry policy and timeouts.	Job worker becomes a Temporal Activity, with retries expressed as a RetryPolicy. Side-effect class not yet annotated — confirm whether this job is safe to retry.	Medium	<i>incl. elsewhere</i>	—
activity-implementation: check-identity-bureau	check-identity-bureau	Implement the Temporal Activity for this job type and set its retry policy and timeouts.	Job worker becomes a Temporal Activity, with retries expressed as a RetryPolicy. Side-effect class not yet annotated — confirm whether this job is safe to retry.	Medium	<i>incl. elsewhere</i>	—
activity-implementation: classify-transactions-dmn	classify-transactions-dmn	Implement the Temporal Activity for this job type and set its retry policy and timeouts.	Job worker becomes a Temporal Activity, with retries expressed as a RetryPolicy. Side-effect class not yet annotated — confirm whether this job is safe to retry.	Medium	<i>incl. elsewhere</i>	—

activity-implementation: confirm-fx-settlement	confirm-fx-settlement	Implement the Temporal Activity for this job type and set its retry policy and timeouts.	Job worker becomes a Temporal Activity, with retries expressed as a RetryPolicy. Side-effect class not yet annotated — confirm whether this job is safe to retry.	Medium	<i>incl. elsewhere</i>	—
activity-implementation: core-banking-api-call	core-banking-api-call	Implement the Temporal Activity for this job type and set its retry policy and timeouts.	Job worker becomes a Temporal Activity, with retries expressed as a RetryPolicy. Side-effect class not yet annotated — confirm whether this job is safe to retry.	Medium	<i>incl. elsewhere</i>	—
activity-implementation: escalate-sla-breach	escalate-sla-breach	Implement the Temporal Activity for this job type and set its retry policy and timeouts.	Job worker becomes a Temporal Activity, with retries expressed as a RetryPolicy. Side-effect class not yet annotated — confirm whether this job is safe to retry.	Medium	<i>incl. elsewhere</i>	—
activity-implementation: find-affected-clients	find-affected-clients	Implement the Temporal Activity for this job type and set its retry policy and timeouts.	Job worker becomes a Temporal Activity, with retries expressed as a RetryPolicy. Side-effect class not yet annotated — confirm whether this job is safe to retry.	Medium	<i>incl. elsewhere</i>	—

activity-implementation: flag-payment-for-review	flag-payment-for-review	Implement the Temporal Activity for this job type and set its retry policy and timeouts.	Job worker becomes a Temporal Activity, with retries expressed as a RetryPolicy. Side-effect class not yet annotated — confirm whether this job is safe to retry.	Medium	<i>incl. elsewhere</i>	—
activity-implementation: flag-product-book	flag-product-book	Implement the Temporal Activity for this job type and set its retry policy and timeouts.	Job worker becomes a Temporal Activity, with retries expressed as a RetryPolicy. Side-effect class not yet annotated — confirm whether this job is safe to retry.	Medium	<i>incl. elsewhere</i>	—
activity-implementation: handle-case-sla-breach	handle-case-sla-breach	Implement the Temporal Activity for this job type and set its retry policy and timeouts.	Job worker becomes a Temporal Activity, with retries expressed as a RetryPolicy. Side-effect class not yet annotated — confirm whether this job is safe to retry.	Medium	<i>incl. elsewhere</i>	—
activity-implementation: io.camunda:http-json-1	io.camunda:http-json-1	Implement the Temporal Activity for this job type and set its retry policy and timeouts.	Job worker becomes a Temporal Activity, with retries expressed as a RetryPolicy. Side-effect class not yet annotated — confirm whether this job is safe to retry.	Medium	<i>incl. elsewhere</i>	—

activity- implementati on: lock- fx- rate	lock-fx-rate	Implement the Temporal Activity for this job type and set its retry policy and timeouts.	Job worker becomes a Temporal Activity, with retries expressed as a RetryPolicy. Side-effect class not yet annotated — confirm whether this job is safe to retry.	Medium	<i>incl. elsew here</i>	—
activity- implementati on: manual- ledger- adjustment	manual- ledger- adjustment	Implement the Temporal Activity for this job type and set its retry policy and timeouts.	Job worker becomes a Temporal Activity, with retries expressed as a RetryPolicy. Side-effect class not yet annotated — confirm whether this job is safe to retry.	Medium	<i>incl. elsew here</i>	—
activity- implementati on: notify- customer	notify- customer	Implement the Temporal Activity for this job type and set its retry policy and timeouts.	Job worker becomes a Temporal Activity, with retries expressed as a RetryPolicy. Side-effect class not yet annotated — confirm whether this job is safe to retry.	Medium	<i>incl. elsew here</i>	—
activity- implementati on: notify- ledger- posted	notify- ledger- posted	Implement the Temporal Activity for this job type and set its retry policy and timeouts.	Job worker becomes a Temporal Activity, with retries expressed as a RetryPolicy. Side-effect class not yet annotated — confirm whether this job is safe to retry.	Medium	<i>incl. elsew here</i>	—

activity-implementation: notify-senior-analyst	notify-senior-analyst	Implement the Temporal Activity for this job type and set its retry policy and timeouts.	Job worker becomes a Temporal Activity, with retries expressed as a RetryPolicy. Side-effect class not yet annotated — confirm whether this job is safe to retry.	Medium	<i>incl. elsewhere</i>	—
activity-implementation: notify-valuation-appointment	notify-valuation-appointment	Implement the Temporal Activity for this job type and set its retry policy and timeouts.	Job worker becomes a Temporal Activity, with retries expressed as a RetryPolicy. Side-effect class not yet annotated — confirm whether this job is safe to retry.	Medium	<i>incl. elsewhere</i>	—
activity-implementation: process-cancellation	process-cancellation	Implement the Temporal Activity for this job type and set its retry policy and timeouts.	Job worker becomes a Temporal Activity, with retries expressed as a RetryPolicy. Side-effect class not yet annotated — confirm whether this job is safe to retry.	Medium	<i>incl. elsewhere</i>	—
activity-implementation: process-withdrawal	process-withdrawal	Implement the Temporal Activity for this job type and set its retry policy and timeouts.	Job worker becomes a Temporal Activity, with retries expressed as a RetryPolicy. Side-effect class not yet annotated — confirm whether this job is safe to retry.	Medium	<i>incl. elsewhere</i>	—

activity-implementation: reconcile-fx- ledger	reconcile-fx- ledger	Implement the Temporal Activity for this job type and set its retry policy and timeouts.	Job worker becomes a Temporal Activity, with retries expressed as a RetryPolicy. Side-effect class not yet annotated — confirm whether this job is safe to retry.	Medium	<i>incl. elsewhere</i>	—
activity-implementation: record-supplementary- evidence	record-supplementary- evidence	Implement the Temporal Activity for this job type and set its retry policy and timeouts.	Job worker becomes a Temporal Activity, with retries expressed as a RetryPolicy. Side-effect class not yet annotated — confirm whether this job is safe to retry.	Medium	<i>incl. elsewhere</i>	—
activity-implementation: release-fund- reservation	release-fund- reservation	Implement the Temporal Activity for this job type and set its retry policy and timeouts.	Job worker becomes a Temporal Activity, with retries expressed as a RetryPolicy. Side-effect class not yet annotated — confirm whether this job is safe to retry.	Medium	<i>incl. elsewhere</i>	—
activity-implementation: rescreen-sanctions- pep	rescreen-sanctions- pep	Implement the Temporal Activity for this job type and set its retry policy and timeouts.	Job worker becomes a Temporal Activity, with retries expressed as a RetryPolicy. Side-effect class not yet annotated — confirm whether this job is safe to retry.	Medium	<i>incl. elsewhere</i>	—

<code>activity-implementation: reserve-funds</code>	<code>reserve-funds</code>	Implement the Temporal Activity for this job type and set its retry policy and timeouts.	Job worker becomes a Temporal Activity, with retries expressed as a RetryPolicy. Side-effect class not yet annotated — confirm whether this job is safe to retry.	Medium	<i>incl. elsewhere</i>	—
<code>activity-implementation: retry-alternate-rail</code>	<code>retry-alternate-rail</code>	Implement the Temporal Activity for this job type and set its retry policy and timeouts.	Job worker becomes a Temporal Activity, with retries expressed as a RetryPolicy. Side-effect class not yet annotated — confirm whether this job is safe to retry.	Medium	<i>incl. elsewhere</i>	—
<code>activity-implementation: screen-sanctions-pep</code>	<code>screen-sanctions-pep</code>	Implement the Temporal Activity for this job type and set its retry policy and timeouts.	Job worker becomes a Temporal Activity, with retries expressed as a RetryPolicy. Side-effect class not yet annotated — confirm whether this job is safe to retry.	Medium	<i>incl. elsewhere</i>	—
<code>activity-implementation: send-complaint-status-update</code>	<code>send-complaint-status-update</code>	Implement the Temporal Activity for this job type and set its retry policy and timeouts.	Job worker becomes a Temporal Activity, with retries expressed as a RetryPolicy. Side-effect class not yet annotated — confirm whether this job is safe to retry.	Medium	<i>incl. elsewhere</i>	—

<code>activity-implementation: send-document-reminder</code>	<code>send-document-reminder</code>	Implement the Temporal Activity for this job type and set its retry policy and timeouts.	Job worker becomes a Temporal Activity, with retries expressed as a RetryPolicy. Side-effect class not yet annotated — confirm whether this job is safe to retry.	Medium	<i>incl. elsewhere</i>	—
<code>activity-implementation: settle-fx-leg</code>	<code>settle-fx-leg</code>	Implement the Temporal Activity for this job type and set its retry policy and timeouts.	Job worker becomes a Temporal Activity, with retries expressed as a RetryPolicy. Side-effect class not yet annotated — confirm whether this job is safe to retry.	Medium	<i>incl. elsewhere</i>	—
<code>activity-implementation: submit-payment-rail</code>	<code>submit-payment-rail</code>	Implement the Temporal Activity for this job type and set its retry policy and timeouts.	Job worker becomes a Temporal Activity, with retries expressed as a RetryPolicy. Side-effect class not yet annotated — confirm whether this job is safe to retry.	Medium	<i>incl. elsewhere</i>	—

Platform dependency (customer-side) (3)

Runtime components Temporal does not ship. Sized separately, not included above.

ID	Location	What to do	Why	Complexity	Size	Blocked by
<code>platform-dependency: connector-activity-library</code>	<code>Connector Activity Library</code>	Customer-side platform build — scoped and sized separately, not included in the sizes above.	2 distinct connector type(s) across 2 usage(s) need Activity equivalents.	High	<i>sized separately</i>	—
<code>platform-dependency: decision-service-dmn-engine-replacement</code>	<code>Decision service (DMN engine replacement)</code>	Customer-side platform build — scoped and sized separately, not included in the sizes above.	5 DMN decision(s) need an evaluation engine.	High	<i>sized separately</i>	—

platform- dependency: user- task- service- tasklist- replacement	User task service (Tasklist replacement)	Customer-side platform build — scoped and sized separately, not included in the sizes above.	6 user task(s) across 5 form(s) depend on a task service Temporal does not provide.	High	sized separately	—
---	---	---	---	------	---------------------	---

Size key — a number is this item's own weight, counted toward the estate total. *incl. elsewhere*: this work is real but its weight is already counted in another row (usually that process's workflow-translation item) — not free, just not counted twice. *not sized*: no rubric weight exists for this kind of work yet. *sized separately*: a customer-side platform build, deliberately excluded from the estate total and scoped on its own.